

LongCat-Flash Technical Report

Meituan LongCat Team
longcat-team@meituan.com

ABSTRACT

We introduce LongCat-Flash, a 560-billion-parameter Mixture-of-Experts (MoE) language model designed for both computational efficiency and advanced agentic capabilities. Stemming from the need for scalable efficiency, LongCat-Flash adopts two novel designs: (a) *Zero-computation Experts*, which enables dynamic computational budget allocation and activates 18.6B–31.3B (27B on average) per token depending on contextual demands, optimizing resource usage. (b) *Shortcut-connected MoE*, which enlarges the *computation-communication overlap window*, demonstrating notable gains in inference efficiency and throughput compared to models of a comparable scale. We develop a comprehensive scaling framework for large models that combines hyperparameter transfer, model-growth initialization, a multi-pronged stability suite, and deterministic computation to achieve stable and reproducible training. Notably, leveraging the synergy among scalable architectural design and infrastructure efforts, we complete model training on more than 20 trillion tokens within 30 days, while achieving *over 100 tokens per second (TPS) for inference at a cost of \$0.70 per million output tokens*. To cultivate LongCat-Flash towards *agentic intelligence*, we conduct a large-scale pre-training on optimized mixtures, followed by targeted mid- and post-training on reasoning, code, and instructions, with further augmentation from synthetic data and tool use tasks. Comprehensive evaluations demonstrate that, as a non-thinking foundation model, LongCat-Flash delivers highly competitive performance among other leading models, with exceptional strengths in agentic tasks. The model checkpoint of LongCat-Flash is open-sourced to foster community research.

LongCat Chat: <https://longcat.ai>

Hugging Face: <https://huggingface.co/meituan-longcat>

GitHub: <https://github.com/meituan-longcat>

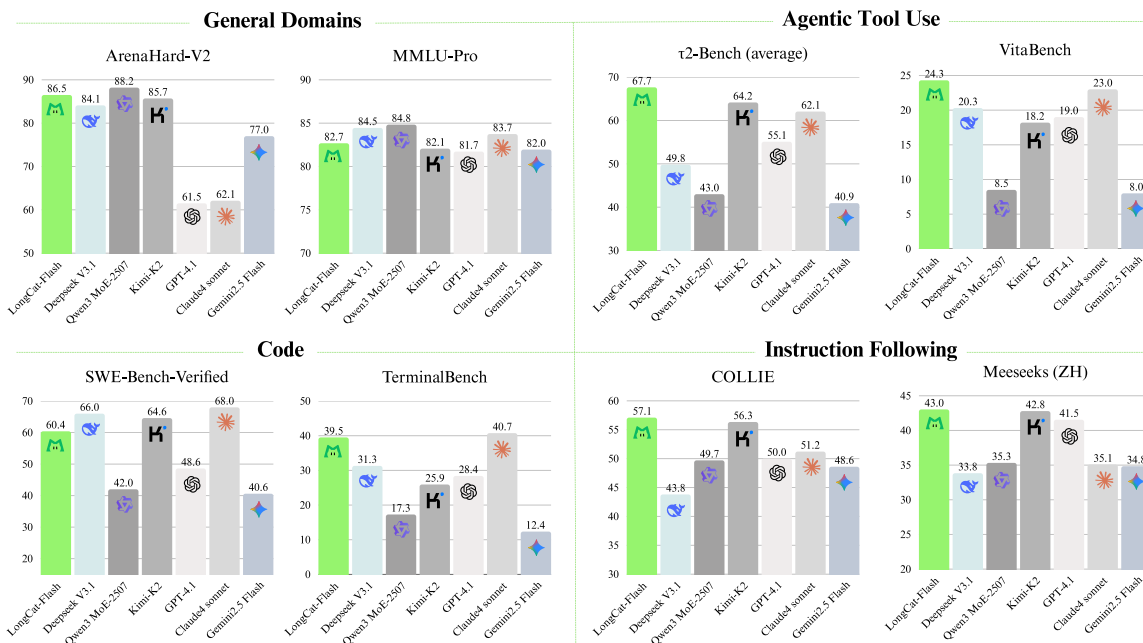


Figure 1: Benchmark performance of LongCat-Flash.

Contents

1	Introduction	4
2	Architecture	5
2.1	Zero-Computation Experts	5
2.1.1	Computational Budget Control	6
2.1.2	Load Balance Control	7
2.2	Shortcut-Connected MoE	7
2.3	Variance Alignment Design for Scalability	8
2.3.1	Scale-Correction for MLA	8
2.3.2	Variance Compensation for Experts Initialization	9
2.4	Model Information	10
3	Pre-Training	10
3.1	Training Strategy	10
3.1.1	Hyperparameter Transfer	10
3.1.2	Model Growth Initialization	11
3.1.3	Training Stability	11
3.2	General Pre-Training	13
3.3	Reasoning and Coding Enhancement	13
3.4	Long Context Extension	14
3.5	Decontamination	14
3.6	Evaluation	14
3.6.1	Evaluation Benchmarks and Configurations	14
3.6.2	Evaluation Results	15
4	Post-Training	15
4.1	Reasoning and Coding	16
4.2	Agentic Tool Use	16
4.3	General Capability	17
4.4	Evaluation	18
4.4.1	Evaluation Benchmarks and Configurations	18
4.4.2	Evaluation Results	19
5	Training Infrastructures	20
5.1	Numerical Precision Control and Fault Detection	21
5.2	Kernel Optimization for Determinism and Performance	21
5.3	Distributed Strategy for Large-scale Training	22
5.4	Reliability and Observability	23
6	Inference and Deployment	23

6.1	Model-Specific Inference Optimization	23
6.1.1	Computation and Communication Orchestration	24
6.1.2	Speculative Decoding	24
6.1.3	Reducing KV Cache	24
6.2	System-Wide Inference Techniques	25
6.2.1	Minimize Schedule Overhead	25
6.2.2	Custom Kernel	25
6.2.3	Quantization	26
6.3	Deployment and Performance	26
6.3.1	Measured Performance	26
6.3.2	Theoretical Performance	27
7	Conclusion	28
8	Contributions	29
A	Appendix	35
A.1	Statistics and Case Studies of Dynamic Routing	35

1 Introduction

The rapid advancement of large language models (LLMs) such as DeepSeek-V3 [DeepSeek-AI et al., 2025], Qwen 3 [Yang et al., 2025], and Kimi-K2 [Team et al., 2025] has demonstrated the effectiveness of scaling model size and computational resources. While some recent progress raises concerns about potential scaling slowdowns, we believe that algorithmic design, underlying system optimizations, and data strategy all play equally critical roles in further pushing the frontier of scalable intelligence. This requires innovations in both model architecture and training strategies to improve the cost-effectiveness of scaling, as well as a systematic data strategy to enhance the model’s capability for solving real-world tasks.

In this work, we introduce LongCat-Flash, an efficient yet powerful Mixture-of-Experts (MoE) language model designed to advance the frontier of language model along two synergistic directions: *computational efficiency* and *agentic capability*. Trained on tens of thousands of accelerators, LongCat-Flash combines architectural innovations with a sophisticated, multi-stage training methodology for scalable and intelligent models. Our contributions span both efficiency and agentic intelligence:

- Scalable Architectural Design for Computational Efficiency** LongCat-Flash is designed and optimized under two key principles: efficient computation utilization, as well as efficient training and inference. Specifically, (1) As *not all tokens are equal*, we introduce the *zero-computation experts* mechanism in MoE blocks to *allocate a dynamic computation budget* to important tokens based on their significance, i.e., activating 18.6 to 31.3 billion parameters (out of 560 billion total) based on contextual demands. To ensure consistent computation load, we employ expert bias adjusted by a *PID-controller*, maintaining an average of ~ 27 billion activated parameters per token. (2) As communication overhead becomes a bottleneck during MoE model scaling, we incorporate the *Shortcut-connected MoE (ScMoE)* [Cai et al., 2024] design to expand the computation-communication overlap window. Combined with customized infrastructure optimizations, this design enables training at a massive scale of over tens of thousands of accelerators and inference with high throughput and low latency.
- Effective Model Scaling Strategy** Effectively and efficiently scaling model size remains a key challenge in strategy design. To this end, we develop a *comprehensive stability-and-scaling framework* for robustly training large-scale models: (1) We successfully apply a hyperparameter transfer strategy to such a large model, predicting optimal hyperparameter configurations by leveraging results from smaller proxy models with theoretical guarantees. (2) We initialize the model using a model-growth mechanism based on a refined half-scale checkpoint, achieving improved performance compared to conventional initialization methods. (3) A multi-pronged stability suite incorporates principled router-gradient balancing, a hidden z-loss to suppress massive activations, and fine-tuned optimizer configurations. (4) To enhance the reliability of large-scale cluster training, we introduce deterministic computation. This guarantees the exact reproducibility of experiments and enables the detection of SDC (Silent Data Corruption) during the training process. These interventions ensure that LongCat-Flash’s training remains stable, with no irrecoverable loss spikes.
- Multi-Stage Training Pipeline for Agentic Capability** Through a meticulously designed pipeline, LongCat-Flash is endowed with advanced agentic behaviors. Initial efforts focus on constructing a more suitable base model for agentic post-training, where we design a two-stage pretraining data fusion strategy to concentrate reasoning-intensive domain data. During mid-training, we enhance reasoning and coding capabilities while extending the *context length to 128k* to meet agentic post-training requirements. Building on this advanced base model, we proceed with a multi-stage post-training. Recognizing the scarcity of high-quality, high-difficulty training problems for agentic tasks, we design a *multi-agent synthesis framework* that defines task difficulty across three axes, i.e., information processing, tool-set complexity, and user interaction—using specialized controllers to generate complex tasks requiring iterative reasoning and environmental interaction.

Overall, benefiting from our synergy among scalable architectural design, training strategies, and infrastructure efforts, LongCat-Flash achieves both high training throughput and low inference latency. Notably, we complete the pre-training of our 560B model over 20T tokens within 30 days and achieve 98.48% time availability without manual intervention for fault resolution. During inference, large-scale deployment efficiency exceeds 100 tokens per second (TPS) on H800, with a cost of \$0.7 per million output tokens, demonstrating remarkable performance compared to models with similar size.

We evaluate the base and instruction-tuned versions of LongCat-Flash across diverse benchmarks, with an overview summarized in Figure 1. As a non-thinking model, LongCat-Flash achieves performance comparable to state-of-the-art non-thinking models, including DeepSeek-V3.1 [DeepSeek-AI et al., 2025] and Kimi-K2 [Team et al., 2025], while using fewer parameters and offering faster inference speed. Specifically, LongCat-Flash scores 86.5 on ArenaHard-V2, 39.5 on TerminalBench, and 67.7 on τ^2 -Bench, demonstrating robust capabilities in general domains, coding, and agentic tool use. To mitigate potential contamination from existing open-source benchmarks and enhance evaluation confidence,

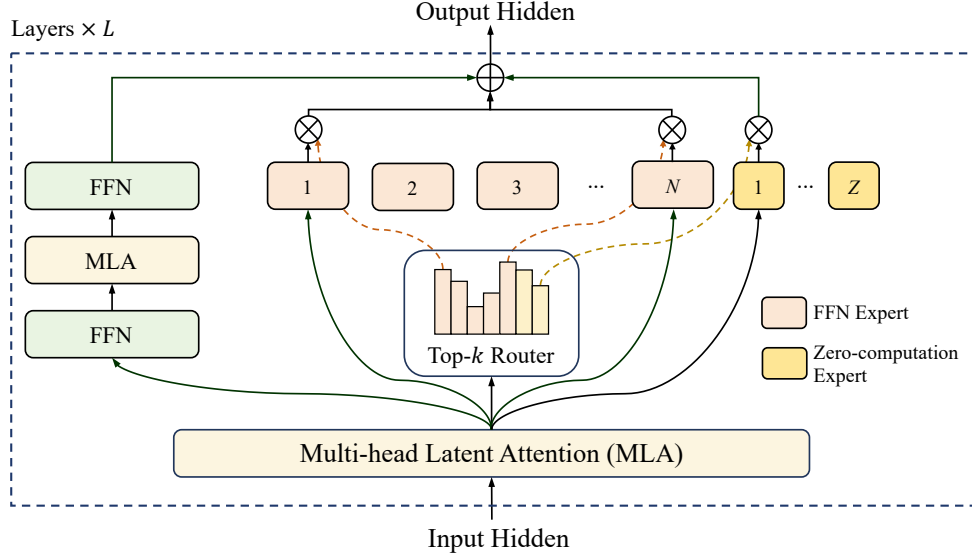


Figure 2: The architecture adopted in LongCat-Flash. Each layer employs Shortcut-connected Mixture of Experts (ScMoE) with zero-computation experts. ScMoE significantly expands the computation-communication window to boost training and inference efficiency. The zero-computation experts enable dynamic computation based on contextual importance, improving the efficiency of computational resource utilization.

we meticulously constructed two new benchmarks: **Meeseeks** [Wang et al., 2025a] and **VitaBench**. Meeseeks simulates realistic human-LLM interactions through an iterative feedback framework to evaluate multi-turn instruction-following ability, where LongCat-Flash achieves scores on par with frontier LLMs. VitaBench leverages real-world business scenarios to access models’ proficiency in addressing complex real-world tasks, where LongCat-Flash delivers superior performance than other LLMs.

In the remainder of this report, we first detail the architecture and innovations in LongCat-Flash. Then, we describe the pre-training and post-training processes, including our training strategies, data construction methods, and evaluation results. Finally, we discuss the challenges and solutions in training LongCat-Flash, along with optimized inference and deployment methods that leverage its unique architecture.

2 Architecture

LongCat-Flash adopts a novel MoE architecture with two key innovations (Figure 2): (1) The MoE block incorporates zero-computation experts [Jin et al., 2024] to enable dynamic computation, allowing tokens to consume variable computational resources based on their contextual significance. Furthermore, the average computational load is regulated through an adaptive expert bias. (2) Each layer integrates two Multi-head Latent Attention (MLA) block [Liu et al., 2024a] and multiple heterogeneous Feed-Forward Network (FFN) blocks. A *shortcut* connection from the first MLA output directly to the MoE block [Cai et al., 2024] is employed. To further enhance performance, we refine both the MLA and fine-grained FFN experts via variance alignment. The following subsections will detail each of these components.

2.1 Zero-Computation Experts

Next-token prediction exhibits inherent computational heterogeneity. Difficult tokens may demand more resources for accurate prediction, while easy tokens require negligible computation. This phenomenon is also empirically evidenced by speculative decoding, where small draft models reliably predict the outputs of large models for most easy tokens [Leviathan et al., 2023].

Motivated by this, LongCat-Flash presents a dynamical computational resource allocation mechanism by **activating a variable number of FFN experts per token** through zero-computation experts [Jin et al., 2024, Zeng et al., 2024], enabling a more reasonable allocation of computations according to contextual significance. Specifically, LongCat-Flash expands its expert pool with Z zero-computation experts in addition to N standard FFN experts. Zero-computation

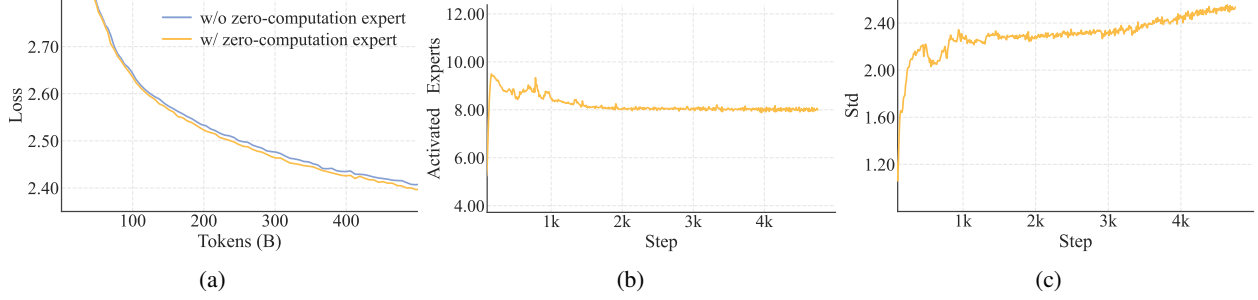


Figure 3: **(a)** Validation loss curve comparing models with/without zero-computation experts under matched computation budgets. The baseline (top-k=8, blue) activates fixed 6B parameters per token, while the zero-expert variant (top-k=12, orange) dynamically activates 4.2B-7.0B parameters but maintains 8 FFN experts expectation (with fluctuation less than 1%). The consistent loss reduction demonstrates the efficacy of zero-computation experts. **(b)** The average number of activated FFN experts during LongCat-Flash training. The average number remains closely around 8, corresponding to expected 27B activated parameters. **(c)** The standard deviation of activated FFN experts grows to 3, indicating substantial variability in activated parameters across different tokens.

experts simply return the input x_t as their output, thereby introducing no additional computational cost. Let x_t be the MoE input of the t -th token, the MoE module in LongCat-Flash can be formulated as follows:

$$\begin{aligned} \text{MoE}(x_t) &= \sum_{i=1}^{N+Z} g_i E_i(x_t), \\ g_i &= \begin{cases} R(x_t)_i, & \text{if } R(x_t)_i \in \text{TopK}(R(x_t)_i + b_i \mid 1 \leq i \leq N + Z, K), \\ 0, & \text{otherwise,} \end{cases} \\ E_i(x_t) &= \begin{cases} \text{FFN}_i(x_t), & \text{if } 1 \leq i \leq N, \\ x_t, & \text{if } N < i \leq N + Z, \end{cases} \end{aligned} \quad (1)$$

where R denotes the softmax router, b_i is the expert bias corresponding to the i -th expert, and K denotes the number of experts selected per token.

The router assigns each token to K experts, where the number of activated FFN experts varies per token based on contextual importance. Through this adaptive allocation mechanism, the model learns to dynamically allocate more computational resources to tokens with higher contextual importance, thus achieving superior performance under the same computational capacity as illustrated in Figure 3a.

2.1.1 Computational Budget Control

To incentivize the model to learn context-dependent computation allocation, fine-grained control over the average selection ratio of zero-computation experts is essential. Without explicit constraints, the model tends to under-utilize zero-computation experts, leading to inefficient resource usage.

We accomplish this by refining the expert bias mechanism from the aux-loss-free strategy [Wang et al., 2024a], introducing an expert-specific bias term that dynamically adjusts routing scores based on recent expert utilization, while remaining decoupled from the language model (LM) training objective. For the expert bias b_i corresponding to the i -th expert, it is updated each step with the incremental computed as:

$$\Delta b_i = \begin{cases} \mu \left(\frac{K_e}{K} \cdot \frac{1}{N} - \frac{T_i}{K T_{\text{all}}} \right), & \text{if } 1 \leq i \leq N, \\ 0, & \text{if } N < i \leq N + Z, \end{cases} \quad (2)$$

where μ denotes the bias adaptation rate, T_{all} denotes the number of tokens in a global batch, T_i denotes the number of tokens routed to the i -th expert, K_e denotes the expected number of activated FFN experts, which is smaller than K .

The proposed update rule employs a PID controller (proportional-integral-derivative) from control theory [Bennett, 1993], ensuring that the token allocation for the i -th expert converges to its target proportion. Compared to a fixed bias

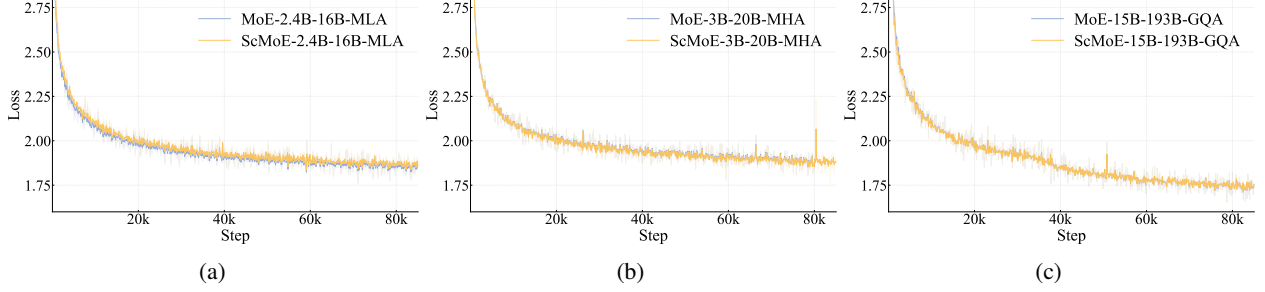


Figure 4: Training loss curves comparing baseline models (without ScMoE) against their ScMoE-enhanced counterparts across four different model configurations. In all experiments—(a) 2.4B-16B with MLA, (b) 3B-20B with MHA, and (c) 15B-193B with GQA—the loss curves are virtually indistinguishable. This provides robust evidence that the ScMoE optimization is quality-neutral, and its benefits are orthogonal to both model scale and the specific attention architecture used.

increment [Wang et al., 2024a], this mechanism improves the robustness of the softmax router’s probability distribution as the number of experts scales. Notably, we exclude zero-computation experts from bias updates, as their identity nature only requires a global constraint, which is automatically satisfied when all FFN experts achieve their expected token proportions. Empirically, large batch sizes and a decay schedule for μ improve the stability of budget control, while small batch sizes may require reduced update frequency.

During pre-training, we tracked the average number and standard deviation of activated experts (Figure 3b and 3c). The results show that after approximate 20B tokens of adjustment, the average expert number in all layers converged to the expected value, with fluctuations less than 1%. However, the standard deviation persisted at a relatively high level, indicating that the model allocates substantially divergent computational resources across tokens.

For detailed statistics and case studies of dynamic routing, please refer to Appendix A.1.

2.1.2 Load Balance Control

Efficient MoE training requires robust load balancing among FFN experts. While Eq. (2) enforces balance at the corpus level, a device-level load balance loss [DeepSeek-AI et al., 2025] to further prevent extreme sequence-level imbalance among EP groups is introduced. We make necessary efforts to accommodate the zero-computation experts. Specifically, assuming that all N FFN experts are divided into D groups, each group containing $G = \frac{N}{D}$ experts, the loss can be expressed as:

$$\mathcal{L}_{LB} = \alpha \sum_{j=1}^{D+1} f_j P_j, \quad (3)$$

$$P_j = \frac{1}{T} \sum_{i \in \text{Group}_j} \sum_{t=1}^T R(x_t)_i, \quad (4)$$

$$f_j = \begin{cases} \frac{D}{K_e T} \sum_{t=1}^T \mathbb{I}(\text{token } t \text{ selects Group}_j), & \text{if } 1 \leq j \leq D, \\ \frac{1}{(K - K_e) T} \sum_{t=1}^T \mathbb{I}(\text{token } t \text{ selects zero-computation experts}), & \text{if } j = D + 1, \end{cases} \quad (5)$$

where α is the balance factor, T is the number of tokens in a micro batch, and $\mathbb{I}$ denotes the indicator function. In the loss, we assign all zero-computation experts to an additional group and average the frequency in each group. By adjusting the coefficient of f_j , we ensure that the ratio of FFN experts to zero-computation experts would approach $\frac{K_e}{K - K_e}$ when the loss converges.

2.2 Shortcut-Connected MoE

Our initial architecture employs an interleaved topology of MoE and dense FFN blocks. This design has been extensively validated through empirical studies, demonstrating performance comparable to leading shared-expert

models [Rajbhandari et al., 2022, Liu et al., 2024a]. However, the efficiency of large-scale MoE models is largely constrained by communication overhead. In the conventional execution paradigm, expert parallelism imposes a sequential workflow: an collective operation must first route tokens to their designated experts before computation can begin. This communication latency becomes a bottleneck, leading to device underutilization and limiting overall system throughput.

While shared-expert architectures attempt to mitigate this by overlapping communication with a single expert’s computation, their efficiency is limited by the small computational window of that one expert. We overcome this limitation by employing the Shortcut-connected MoE (ScMoE) architecture [Cai et al., 2024]. ScMoE introduces a cross-layer shortcut that reorders the execution pipeline. This key innovation allows the dense FFN from the preceding block to execute in parallel with the dispatch/combine communication of the current MoE layer, creating a more substantial overlap window than shared-expert designs. Furthermore, the architecture design choice is verified by the following key findings.

First, ScMoE structure does not compromise model quality. As shown in Figure 4, the training loss curves of our architecture and the baseline without ScMoE are nearly identical, confirming this reordered execution does not impair model performance. Consistent results are observed across multiple settings, including a 2.4B-16B MoE model with MLA, a 3B-20B model with MHA [Vaswani et al., 2017], and 15B-193B models with GQA [Ainslie et al., 2023]. Importantly, these findings demonstrate that the stability and benefits of ScMoE are orthogonal to the choice of attention mechanism.

Second, the ScMoE architecture delivers substantial system-level efficiency gains for both training and inference.

For Large-Scale Training: The expanded overlap window allows the computation of the preceding block to be fully parallel with its dispatch and combine communication phases in the MoE layer, achieved by partitioning operations into fine-grained chunks along the token dimension.

For Efficient Inference: ScMoE enables a *Single Batch Overlap* pipeline, reducing the theoretical Time-Per-Output-Token (TPOT) by nearly 50% compared to leading models such as DeepSeek-V3. Moreover, it allows for the concurrent execution of distinct communication patterns: intra-node Tensor Parallelism communication (via NVLink) on the dense FFN can be fully overlapped with inter-node Expert Parallelism communication (via RDMA), thereby maximizing total network utilization.

In summary, ScMoE delivers substantial performance gains without sacrificing model quality. These efficiency gains are not achieved through trade-offs but are the direct outcome of a rigorously validated, quality-neutral architectural innovation.

2.3 Variance Alignment Design for Scalability

Architectural designs that excel at small scales may become suboptimal as models are scaled up, and vice versa, rendering initial design choices unreliable. Through extensive experimentation and theoretical analysis, we identify *variance misalignment* in specific modules as a key factor contributing to this discrepancy, which can lead to instability and degraded performance during scaling. To address this challenge, we propose variance alignment techniques for both MLA and MoE blocks.

2.3.1 Scale-Correction for MLA

LongCat-Flash employs a modified Multi-head Latent Attention (MLA) mechanism [Liu et al., 2024a], which incorporates scale-correction factors α_q and α_{kv} to address the variance imbalances inherent in asymmetric low-rank factorization. Our full mathematical formulation, which integrates these correction factors, is given as follows:

$$\begin{aligned}
 c_t^Q &= [\alpha_q] W^{DQ} h_t \in \mathbb{R}^{d_q}, & c_t^{KV} &= [\alpha_{kv}] W^{DKV} h_t \in \mathbb{R}^{d_{kv}}, \\
 q_{t,1:n_h}^C &= W^{UQ} c_t^Q, & k_{t,1:n_h}^C &= W^{UK} c_t^{KV}, & v_{t,1:n_h} &= W^{UV} c_t^{KV}, \\
 q_{t,1:n_h}^R &= \text{RoPE}(W^{QR} c_t^Q), & k_{t,1:n_h}^R &= \text{RoPE}(W^{KR} h_t), \\
 q_{t,i} &= [q_{t,i}^C; q_{t,i}^R], & k_{t,i} &= [k_{t,i}^C; k_{t,i}^R], \\
 o_{t,i} &= \text{Attention}(q_{t,i}, k_{1:t,i}, v_{1:t,i}), & u_t &= W^O [o_{t,1}; o_{t,2}; \dots; o_{t,n_h}],
 \end{aligned} \tag{6}$$

where $h_t \in \mathbb{R}^{d_{\text{model}}}$ is the attention input of the t -th token, and n_h is the number of heads.

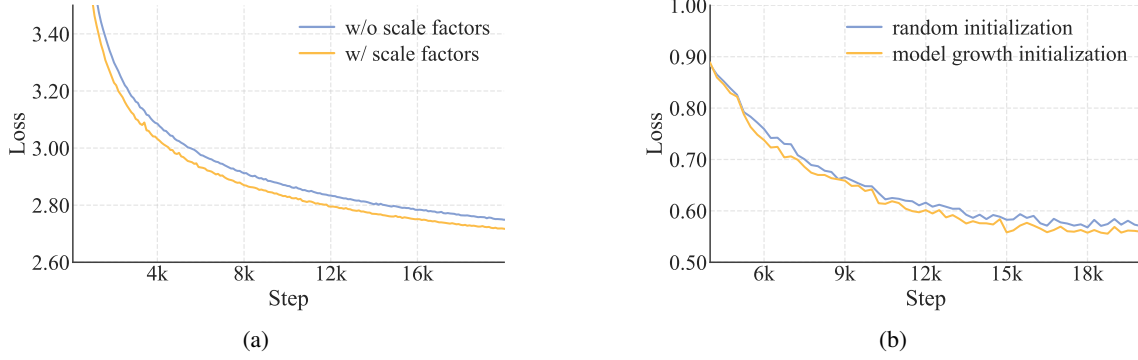


Figure 5: **(a)** Incorporating the scale-correction factor on MLA showing improved convergence (lower loss) on a 1B activated MOE model. **(b)** Validation loss curve of a 6B activated MoE model in model growth experiments.

The introduction of α_q and α_{kv} address a fundamental variance mismatch among query/key vector components. At initialization, their variances are proportional to their source dimensions: $\sigma^2(q_t^C), \sigma^2(q_t^R) \propto d_q$ and $\sigma^2(k_t^C) \propto d_{kv}$. In contrast, the rotary key component k_t^R has a variance proportional to the full model dimension: $\sigma^2(k_t^R) \propto d_{\text{model}}$. This dimensional disparity causes unstable attention scores at initialization when d_q , d_{kv} , and d_{model} are varied, resulting in degraded and unpredictable performance during model scaling.

Our solution is to rescale the low-rank path components to align their final variance with a reference scale, and we use the full model dimension as a reference. This is achieved by defining the scaling factors as:

$$\alpha_q = \sqrt{\frac{d_{\text{model}}}{d_q}} \quad \text{and} \quad \alpha_{kv} = \sqrt{\frac{d_{\text{model}}}{d_{kv}}}. \quad (7)$$

This scale-invariant correction neutralizes the variance mismatch, ensuring they are well-conditioned for the attention computation. Our experiments reveal that this method improves the model performance, as shown in Figure 5a.

2.3.2 Variance Compensation for Experts Initialization

LongCat-Flash adopts the fine-grained expert strategy from DeepSeek-MoE [Liu et al., 2024a], which segments each expert into m finer-grained ones to enhance combinatorial flexibility and knowledge specialization. However, we observe that the performance of this design is sensitive to other architectural choices (e.g., expert numbers, top-k, m). To address this, we propose a variance compensation mechanism that counteracts the initialization variance reduction caused by expert segmentation. The mechanism applies a scaling factor γ to the aggregated output of the experts, formulated as:

$$\text{MoE}(x_t) = \gamma \left(\sum_{i=1}^{mN} g_i \cdot E_i(x_t) \right), \quad (8)$$

where g_i is the router output over mN fine-grained experts and N represents the total number of experts before segmentation.

The scaling factor γ in Eq. (8) is derived by quantifying two primary sources of variance reduction:

1. **Gating Dilution:** Decomposing each original N experts into m finer-grained experts expands the total expert counts to mN . This expansion forces the softmax gate to distribute its probability mass across a larger expert pool, proportionally reducing the magnitude of individual gating values g_i . Consequently, the output variance is reduced approximately by a factor of m .
2. **Dimensional Reduction:** The intermediate hidden dimension of each fine-grained expert ($d_{\text{expert_inter}}$) is reduced by a factor of m . Assuming uniform parameter initialization, the output variance of a single expert also decreases by a factor of m .

To preserve the MoE layer’s output variance at initialization (matching the pre-segmentation baseline), γ must compensate for both effects. The combined variance compensation factor is thus $\gamma = \sqrt{m \cdot m} = m$.

2.4 Model Information

Tokenizer LongCat-Flash employs byte-pair encoding (BPE) [Shibata et al., 1999, Sennrich et al., 2015] for tokenization. Our tokenizer is trained on a comprehensive multilingual corpus spanning web pages, books, source code, etc, ensuring robust cross-domain performance. While inheriting GPT-4’s pre-tokenization framework, we introduce the following modifications: (1) **Enhanced CJK character segmentation for improved Chinese text handling**, and (2) **Independent digit tokenization to boost mathematical capabilities**. The vocabulary size is optimized at 131,072 tokens, striking an effective balance between computational efficiency and linguistic coverage.

Multi-Token Prediction To enhance inference efficiency, we integrate Multi-Token Prediction (MTP) [Gloeckle et al., 2024, DeepSeek-AI et al., 2025] as an auxiliary training objective. For optimal inference performance, we employ a single dense layer rather than a MoE layer as the MTP head. Empirical observations reveal rapid convergence of MTP loss, prompting us to strategically introduce MTP training in the middle of the training pipeline to balance model performance with prediction accuracy. The MTP head achieves >90% acceptance rate in evaluations (Table 5).

Model Configurations LongCat-Flash consists of 28 layers (excluding the MTP layer) with a 6144-dimensional hidden state. Each MLA block uses 64 attention heads with per-head dimension 128 for balanced performance-efficiency tradeoff. Following DeepSeek-V3 [Liu et al., 2024a], we set the KV compression dimension to 512, and the query compression dimension to 1536. The FFNs in the dense path employ 12288 intermediate dimensions, while each FFN expert uses 2048 dimensions. The scaling factors in MLA blocks and FFN blocks follow the methodology in Section 2.3.1. Each layer contains 512 FFN experts and 256 zero-computation experts, with exactly 12 experts activated per token (selected from both types). LongCat-Flash has 560B total parameters, activating between 18.6B and 31.3B parameters per token depending on context, with an average activation of approximately 27B parameters.

3 Pre-Training

The pre-training of LongCat-Flash follows a three-stage curriculum: (1) We train the model on approximately 20 trillion tokens with 8192 sequence length to establish a robust base model. (2) Reasoning and coding capabilities are further enhanced using trillions of data. (3) The context length is extended to 128k through training on long context corpora. Each stage implements tailored data strategies accompanied by rigorous decontamination procedures to prevent test set leakage.

To optimize scalability, we introduce hyperparameter transfer and model growth strategies, significantly improving performance as model size increases. Given the inherent instability challenges in large-scale training, we identify and implement multiple effective techniques to enhance training stability.

3.1 Training Strategy

3.1.1 Hyperparameter Transfer

LongCat-Flash employs a hyperparameter transfer strategy based on width scaling [Everett et al., 2024] to efficiently train large-scale models. The methodology involves: (1) identifying optimal hyperparameters on a smaller proxy model, and (2) transferring these configurations to the target model through theoretically-motivated scaling rules.

The transfer mechanism centers on the width scaling factor $s = n_{\text{target}}/n_{\text{proxy}}$, where n is the model’s hidden dimension. We specifically adopt the “Adam LR Full Align” rules for Standard Parameterization. These rules specify how to adapt the proxy model’s optimal initialization variance (σ^2) and learning rate (η) for the target architecture. The practical transfer rules are summarized in Table 1.

Table 1: Practical hyperparameter transfer rules and their underlying scaling exponents, derived from the Adam LR Full Align principle for Standard Parameterization [Everett et al., 2024]. Here, s is the width scaling factor $n_{\text{target}}/n_{\text{proxy}}$.

Layer & Parameter	Target Model Setting
Embedding (Init Var, σ^2)	$\sigma_{\text{target}}^2 = \sigma_{\text{proxy}}^2$
Embedding (Learning Rate, η)	$\eta_{\text{target}} = \eta_{\text{proxy}}$
Hidden/Unembedding (Init Var, σ^2)	$\sigma_{\text{target}}^2 = \sigma_{\text{proxy}}^2 / s$
Hidden/Unembedding (Learning Rate, η)	$\eta_{\text{target}} = \eta_{\text{proxy}} / s$

Following this methodology, our training involves the following steps:

1. We set the width scaling factor s to 8 based on a trade-off analysis between computational efficiency and transfer performance. The proxy model is configured with a width of 768.
2. We then perform a comprehensive hyperparameter search on the proxy model to identify the optimal layer-specific initialization variances (σ_{proxy}^2) and learning rates (η_{proxy}).
3. The optimal hyperparameters from the proxy model were transferred to the target model following the rules detailed in Table 1. All other architectural attributes (depth, sparsity, and batch size) remain invariant during this transfer process.

We conducted comprehensive experiments to validate the effectiveness of this approach. The results demonstrate that this method significantly reduces computational costs when identifying optimal hyperparameters (initialization variance and learning rate) for large-scale model training, while establishing a robust, theoretically grounded framework for model scaling.

3.1.2 Model Growth Initialization

LongCat-Flash employs model growth as its initialization strategy, starting from a half-scale model pre-trained on tens of billions of tokens. Among existing model growth methods [Chen et al., 2015, Du et al., 2024, Wang et al., 2023a, Shen et al., 2022, Wang et al., 2023b, Gong et al., 2019], we adopt the layer stacking technique [Du et al., 2024, Kim et al., 2023] to expand parameters and enhance performance. Disregarding the embedding and unembedding processes temporarily, the whole procedure is formulated as:

$$L_{\text{small}} = l_1 \circ l_2 \circ \dots \circ l_n$$

$$L_{\text{target}} = \underbrace{L_{\text{small}} \circ L_{\text{small}} \circ \dots \circ L_{\text{small}}}_r$$

where l_i denotes the transformation of the i th layer in the model, r denotes the expansion rate, L_{small} denotes the small model’s transformation from token embeddings to final hidden states, and L_{target} represents the transformation of the target (large) model constructed by stacking r copies of the small model. We use $r = 2$ for our architecture.

Through extensive experiments, we consistently observed that models initialized via model growth exhibit a characteristic loss trajectory: an initial increase followed by accelerated convergence, ultimately outperforming randomly initialized baselines. Figure 5b presents a representative case from our 6B activated model experiments, demonstrating the advantage of model growth initialization.

We conjecture that this improvement arises from two synergistic factors: (1) the faster convergence of smaller models likely provides higher-quality parameter initializations for scaled training, and (2) growth operations potentially serve as implicit regularization against parameter collapse. Experimental evidence further suggests that over-optimizing predecessor models may negatively impact token efficiency in target models, indicating the need for judicious growth timing.

For LongCat-Flash initialization, we first train a 14-layer model with identical architecture to the target model, using random initialization on the initial data segment. The trained model is then stacked to create a 28-layer checkpoint, preserving all training states including sample counters and learning rate schedules from the predecessor.

3.1.3 Training Stability

We enhance the training stability of LongCat-Flash from three perspectives: router stability, activation stability, and optimizer stability.

Router Stability A fundamental challenge in training MoE models is router stability, which stems from the tension between two competing gradients:

- The language modeling (LM) loss, driving *expert specialization* (assigning tokens to the most suitable experts),
- The auxiliary load balancing (LB) loss, enforcing *routing uniformity* (distributing tokens evenly across experts).

When the LB gradient dominates, router parameters for all experts converge toward similarity, leading to uniform routing decisions regardless of input tokens. This nullifies the benefits of conditional computation and severely degrades model performance.

To diagnose and control this behavior, we propose a monitoring framework with two key metrics:

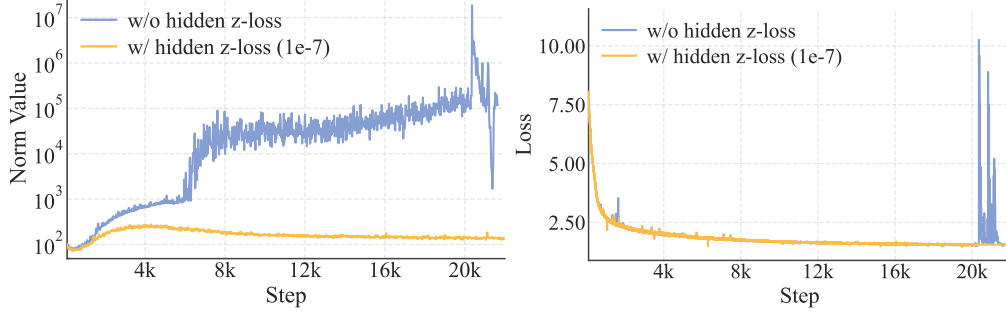


Figure 6: L2 norm of last layer’s hidden states and the training loss for a small model with suboptimal hyper-parameters. The introduction of a negligible-coefficient hidden z-loss stabilizes the norm curve without degrading training loss.

- **Router Weight Similarity:** Measure the average pairwise cosine similarity between expert weight vectors $\{w_i\}$. A high similarity is a direct indicator that the load balancing loss is excessively dominant.
- **Gradient Norm Ratio (R_g):** Quantify the relative influence of the two losses on the batch-averaged expert probability vector $\vec{P}$:

$$R_g = \frac{\|\alpha \nabla_{\vec{P}} \mathcal{L}_{LB}\|_2}{\|\nabla_{\vec{P}} \mathcal{L}_{LM}\|_2}, \quad (9)$$

where $\mathcal{L}_{LB}$ is the load balancing loss computed without the coefficient α .

Guided by this framework, we establish a practical guideline for setting the hyperparameter α . The principle is to ensure the load balancing term acts as a regularizer without overwhelming the LM loss. We therefore recommend choosing a coefficient that keeps the R_g below a small threshold (e.g., $R_g < 0.1$).

Activation Stability via Hidden z-loss Inspired by the router z-loss [Zoph et al., 2022], we design hidden z-loss to circumvent the widespread occurrence of massive activation [Sun et al., 2024] during LLM training. Through empirical observations, we find that such massive activations correlate with severe loss spikes during training, which are associated with optimization instability and potential performance degradation. Hidden z-loss is mainly used to suppress elements with extremely large magnitudes:

$$\mathcal{L}_Z = \frac{\lambda}{T} \sum_{t=1}^T \left(\log \sum_{i=1}^{|z_t|} \exp(\text{abs}(z_t^i)) \right)^2, \quad (10)$$

where λ is the coefficient to weight this loss, z_t is the final layer output of the t -th token (before final layer norm), $|z_t|$ is the hidden state size, and $\text{abs}(\cdot)$ denotes absolute value function. As depicted in Figure 6, we found that a very small loss coefficient can significantly suppress the massive activation phenomenon without compromising training loss, thus reducing the risk of numerical errors during BF16 training.

On the Practical Configuration of Adam’s Epsilon As model scale increases, the epsilon (ε) parameter in the Adam optimizer, traditionally treated as a minor constant for numerical stability, emerges as a critical hyperparameter. OLMo et al. [2024] demonstrated that setting it to 1e-8 yields superior results compared to the default value of 1e-5. This heightened sensitivity primarily stems from two factors: (1) large-scale models typically employ smaller parameter initializations, and (2) they utilize substantially larger batch sizes during training. When using default ε values, the parameter’s magnitude may become comparable to or even exceed the typical scale of gradient second moments, thereby disrupting the optimizer’s adaptive mechanism.

As illustrated in Figure 7, our empirical analysis tracking the Root Mean Square (RMS) norm of gradients reveals two key findings: (1) **Threshold effect:** Significant performance degradation occurs when ε approaches the observed gradient RMS norm; (2) **Lower bound stability:** Once ε is reduced below this critical threshold, further decreases have a negligible impact on model performance. Consequently, we recommend setting ε to a small value (several orders of magnitude smaller than the expected gradient RMS norm). In LongCat-Flash, we adopt $\varepsilon=1\text{e-}16$, a configuration that ensures numerical stability while preserving the optimizer’s adaptive properties.

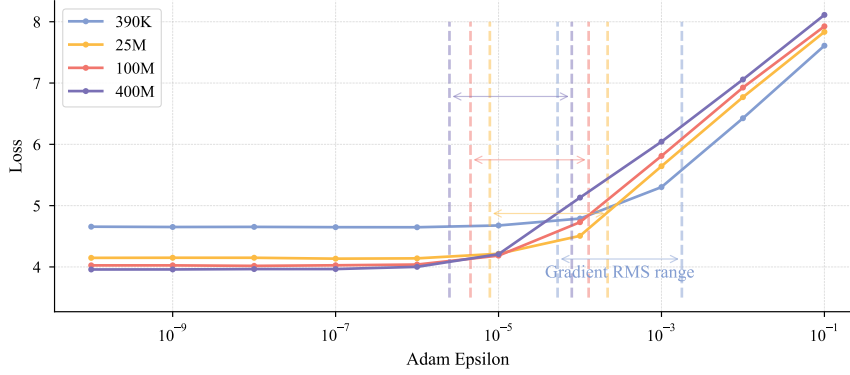


Figure 7: Exploring the impact of the Root Mean Square (RMS) norm of gradients and epsilon on loss across different model sizes. The “Gradient RMS range” denotes the span between the maximum and minimum gradient RMS values for different weights in the model. As the model size increases (ranging from 390K to 400M parameters), the gradient RMS becomes smaller. When epsilon approaches the range of the gradient RMS, a rapid deterioration in loss is observed.

3.2 General Pre-Training

We first conduct a general pre-training stage to ensure overall model ability. A multi-phase pipeline is designed to ensure data quality and diversity. The main phases include:

Content Extraction We utilize a customized version of *trafilatura* [Barbaresi, 2021] for general web content and a dedicated process for STEM material to correctly parse complex elements like formulas, code, and tables.

Quality Filtering A two-step filtering approach is applied. An initial classifier removes clearly low-quality documents, followed by finer-grained screening based on metrics like fluency and content completeness.

Deduplication We apply an efficient MinHash implementation for large-scale deduplication, supplemented by a strategy to identify and handle repetitive web templates for more accurate document-level deduplication.

The final data mixture process adopts a two-stage schedule, progressively increasing the proportion of high-quality reasoning data (e.g., STEM and code).

- Stage 1: For general-purpose data, we employ an instance-level data mixing strategy that balances data quality and diversity described in SampleMix [Xi et al., 2025], where we compute an initial sampling distribution using quality and diversity scores, and further adjust the tendency of the distribution based on fine-grained domain and writing style labels. Redundant low-value domains (e.g., advertisement, sports, hiring) are downsampled, while reasoning-rich domains (e.g., science) are upsampled.
- Stage 2: We prioritize reasoning-intensive domains in this phase, with STEM and code comprising 70% of the final mixture. Preliminary experiments showed that abrupt reductions in general-domain data temporarily degrade model capabilities. Thus, we implement gradual code proportion increases, guided by continuous perplexity monitoring on external validation sets to ensure smooth transitions without compromising general performance.

3.3 Reasoning and Coding Enhancement

To further enhance the model’s reasoning and coding capabilities and establish a robust base model with substantial potential for subsequent post-training, we conduct a mid-training stage utilizing high-quality relevant data generated through a combination of pretraining data retrieval and data synthesis.

The systematic synthetic data workflow is introduced to optimize data quality and diversity through three key mechanisms: (1) Knowledge graph traversal and node combination to ensure conceptual complexity and domain coverage; (2) Multi-stage iterative refinement to progressively improve difficulty levels and Chain-of-Thought (CoT) reasoning quality; (3) Dual-modality generation and verification (textual and computational) to guarantee mathematical accuracy and solution validity. Careful quality control is conducted combining both rule-based and model-based filters, and the final dataset comprises hundreds of billions of tokens.

3.4 Long Context Extension

We implement a two-stage context length extension strategy to meet the requirements for subsequent long-context reasoning and agentic training. In the first stage, the context window expands from 8k to 32k tokens using 80B training tokens, with RoPE’s base [Su et al., 2024] frequency raised from 1,000,000 to 5,000,000. In the second stage, we further extend it to 128k tokens through an additional 20B tokens, increasing the base frequency to 10,000,000.

The training corpus is built upon naturally occurring long-text data, such as high-quality books and novels. Additionally, we developed a systematic approach to organize repository-level source code to improve the model’s long-context capabilities. We carefully selected high-quality repositories and applied a multi-stage filtering process to remove non-textual content, build artifacts, and auto-generated code, resulting in a curated 20B-token dataset for long-context pre-training.

To ensure that the model’s general capabilities remain stable during the length extension, we adopt a data mixture strategy identical to that of our main pre-training phase and augment this mixture with an additional 25% of long-context data to enhance the model’s long-context performance.

3.5 Decontamination

We perform rigorous decontamination on all training data to prevent data leakage from test sets of common benchmarks. For web and code data, we remove documents containing any 13-gram overlap with predefined test sets. For synthetic data and question-answering pairs, we employ a stricter strategy based on semantic similarity using BGE-m3 [Chen et al., 2024] embeddings. Documents are discarded if they meet either of the following criteria: (1) Semantic similarity score > 0.9 to any test case; (2) Lexical overlap (measured by sparse embeddings) combined with a similarity score between 0.7–0.9.

3.6 Evaluation

This section presents a comprehensive evaluation of the LongCat-Flash base model, including the methodology and results.

3.6.1 Evaluation Benchmarks and Configurations

The model evaluation covers four core capabilities: general tasks, general reasoning, mathematical reasoning, and coding. The benchmarks used for assessment include:

- **General Tasks:** MMLU [Hendrycks et al., 2021a], MMLU-Pro [Wang et al., 2024b], C-Eval [Huang et al., 2023], and CMMLU [Li et al., 2023a].
- **Reasoning Tasks:** GPQA [Rein et al., 2023], SuperGPQA [M-A-P Team, ByteDance., 2025], BBH [Suzgun et al., 2023], PIQA [Bisk et al., 2019], DROP [Dua et al., 2019], CLUEWSC [Xu et al., 2020], and WinoGrande [Sakaguchi et al., 2019].
- **Math Tasks:** GSM8K [Cobbe et al., 2021], MATH [Hendrycks et al., 2021b].
- **Coding Tasks:** MBPP+ [Liu et al., 2024b], HumanEval+ [Liu et al., 2024b], MultiPL-E [Cassano et al., 2022], and CRUXEval [Gu et al., 2024].

We compare the LongCat-Flash base model with state-of-the-art open-source base MoE models, including DeepSeek-V3.1 Base [DeepSeek-AI et al., 2025], Llama-4-Maverick Base [Meta AI, 2025], and Kimi-K2 Base [MoonshotAI, 2025].

To ensure fairness, all models are evaluated under identical pipelines and configurations. For minority results that cannot be reproduced, we directly adopt metrics from public reports and explicitly annotate them in Table 2. The evaluation settings are as follows:

- General/reasoning/math tasks: Use few-shot prompts to guide output format. Performance is measured via accuracy or F1 score.
- HumanEval+ and MBPP+: Follow OpenAI’s recommended setting [Chen et al., 2021].
- MultiPL-E: Follow BigCode Evaluation Harness [Ben Allal et al., 2022].
- CRUXEval: Follow the official configuration¹, employing 2-shots examples.

¹<https://github.com/facebookresearch/cruxeval>

3.6.2 Evaluation Results

Table 2 presents the evaluation results across diverse benchmarks. LongCat-Flash Base model achieves performance on par with state-of-the-art base models despite its compact active/total parameter size. Although Llama-4-Maverick has fewer activated and total parameters, LongCat-Flash Base surpasses both on nearly all benchmarks.

A comparative analysis reveals that LongCat-Flash Base matches DeepSeek-V3.1 Base’s performance across all domains despite containing fewer parameters. While the two models perform similarly in general tasks, LongCat-Flash Base demonstrates a notably advantage on the MMLU-Pro benchmark (featuring challenging questions). For reasoning tasks, LongCat-Flash Base attains a higher average score. In math and coding tasks, it outperforms DeepSeek-V3.1 Base on most benchmarks, with only marginal performance gaps observed on CRUXEval and MultiPL-E. Against Kimi K2 Base, LongCat-Flash Base shows modestly lower performance in general tasks but achieves parity or superiority in reasoning, math, and coding tasks.

These results collectively underscore LongCat-Flash Base’s parameter efficiency, as it delivers competitive or superior performance to larger models across the majority of evaluated benchmarks.

Table 2: Comparison between LongCat-Flash and other base models. Values marked with * are sourced from public reports.

Benchmark	DeepSeek-V3.1 Base	Llama-4-Maverick Base	Kimi-K2 Base	LongCat-Flash Base
Architecture	MoE	MoE	MoE	MoE
# Total Params	671B	402B	1043B	560B
# Activated Params	37B	17B	32B	27B
General Domains				
MMLU _(acc)	87.46	84.41	87.47	87.05
MMLU-Pro _(acc)	59.29	63.90	68.36	70.32
CEval _(acc)	89.33	81.93	91.24	87.73
CMMLU _(acc)	88.21	80.71	90.35	87.19
General Reasoning				
GPQA _(acc)	47.16	48.08	45.89	51.09
SuperGPQA _(acc)	-	40.58*	44.70*	54.19
BBH _(acc)	89.46	87.56	89.19	90.54
DROP _(f1)	80.74	77.44	69.81	78.39
PIQA _(acc)	93.00	90.59	95.10	92.33
WinoGrande _(acc)	83.50	73.32	82.87	85.08
CLUEWSC _(acc)	88.16	88.00	76.32	91.12
Mathematical Reasoning				
GSM8K _(acc)	92.22	84.61	92.27	92.19
MATH _(acc)	61.56	63.34	66.74	64.82
Coding				
MBPP+ _(pass@1)	59.26	70.11	80.49	77.25
HumanEval+ _(pass@1)	67.07	60.37	69.84	65.85
MultiPL-E _(pass@1)	62.00	58.35	59.22	69.25
CRUXEval-I _(pass@1)	65.87	62.00	65.87	71.63
CRUXEval-O _(pass@1)	71.25	64.25	68.75	75.88

4 Post-Training

We implement a conventional multi-stage post-training framework to augment the base model’s performance across diverse domains, ranging from sophisticated reasoning, coding and agentic tool use tasks to general-purpose capabilities. During this process, we observed that the limited availability of high-quality problem sets is a significant bottleneck across all domains. In the subsequent sections, we present key insights derived from our post-training methodology, organized into three distinct phases: (1) Reasoning and coding, (2) Agentic tool use, and (3) General capability.

4.1 Reasoning and Coding

Mathematics To generate high-quality and novel problems, we use a persona [Ge et al., 2024], self-instruct [Wang et al., 2022] paradigm. This process is guided by a comprehensive mathematical framework that spans topics from elementary to advanced levels. We leverage a diverse set of math-related “expert” personas to ask questions, steering LLMs to synthesize queries that cover underrepresented subjects. Each query is structured to elicit Chain-of-Thought (CoT) reasoning, promoting step-by-step problem-solving in the generated answers. Details of persona curation and answer verification are as follows:

- **Persona Curation:** The personas are constructed from multiple sources: we generate them from our high-quality pretraining data, derive them from existing math queries, and incorporate relevant collections from Persona Hub. Each persona is systematically labeled by its STEM discipline. To ensure maximum diversity and alignment with our subject framework, we use the MinHash algorithm to select the final set of personas for query generation.
- **Answer Verification:** We employ a two-stage process to ensure the accuracy of the synthesized solutions: (1) We generate answers for each problem using several different LLMs and select the most consistent solution as the final answer. (2) We train a generative reward model, specifically enhanced with reasoning data, to automatically score and verify the logical soundness of the problem-solving steps.

Coding We assemble a diverse set of coding queries from multiple sources, including public datasets, queries generated from GitHub code snippets [Wei et al., 2024] and coding-related forums, as well as queries evolved using the Code Evol-Instruct method [Luo et al., 2024]. The data distribution is balanced according to topic diversity and difficulty. Specifically, we train a model to select queries that are clear, consistent, and correct, with sufficient explanatory detail, and implement a filtering pipeline to eliminate responses containing garbled content, repetitive patterns, or logical errors. For software engineering tasks, we curate and validate ten thousands of Docker images containing test cases. Each image is used to verify whether model-generated code can resolve specific issues in the corresponding repository. We develop an agent-based system that leverages various tools to autonomously analyze code structures, identify relevant files, fix bugs, and implement new features. This process yields thousands of successful trajectories that pass all test cases, thereby enhancing the model’s ability to autonomously solve real-world software engineering problems.

Logical Reasoning We construct logical reasoning datasets covering deductive, hypothetical, and inductive reasoning, which include tasks such as LogicPro [Jiang et al., 2025], PODA [Wang et al., 2025b], and Zebra-style logic puzzles. To manage difficulty, we first use the Pass@k metric for an initial balance, then filter out intractable problems where advanced thinking models failed. We also convert multiple-choice questions to a fill-in-the-blank format to mitigate random guessing. The evaluation of responses focused on four key areas: (1) correctness of the final answer; (2) completeness and clarity of reasoning; (3) avoidance of excessive repetition; and (4) consistent use of language.

4.2 Agentic Tool Use

We define agentic tasks as complex problem resolution through systematic environment interaction. In this paradigm, models must iteratively analyze existing information and determine when environmental interaction is needed. Specifically, within the agentic tool utilization framework, the environment comprises user and tools with distinct characteristics. User functions as an autonomous information-providing entity without upstream or downstream dependencies, but exhibit reluctance to be disturbed and non-spontaneous information disclosure. Consequently, models must minimize user queries while employing strategic questioning techniques to elicit maximally precise information when interaction becomes necessary. Tools can be invoked extensively with high frequency, but exhibit intricate interdependencies. From this perspective, excluding domain-specific expertise such as advanced programming capabilities or mathematical computation, we attribute task difficulty escalation to three factors:

- **Information processing complexity** Models must engage in sophisticated reasoning processes to integrate and transform information into required components.
- **Tool set complexity** By modeling the tool set as a directed graph based on intertool dependencies, complexity can be quantitatively characterized by the graph’s node cardinality and edge density.
- **User interaction complexity** Models must learn to engage in multi-round strategic questioning with minimal frequency, adapting to various conversational styles, levels of communication willingness and patterns of information disclosure, thus facilitating effective user interaction while ensuring adequate information acquisition.

Building on these insights, we construct a multi-agent data synthesis framework that generates high-quality challenging tasks by systematically addressing three complexity dimensions critical for agent training: (1) tool set complexity,

(2) information processing complexity, and (3) user interaction complexity. The framework comprises the following specialized agents:

- **UserProfileAgent** Beyond generating fundamental user profiles encompassing personal information and preferences, we further implement controls over user conversational styles, communication willingness levels, and information disclosure patterns to more accurately simulate authentic user interaction scenarios while simultaneously enhancing task complexity.
- **ToolSetAgent** To maximize data diversity and prevent overfitting to specific scenarios, we adopt an approach analogous to Kimi-K2 [Team et al., 2025], enumerating 40 distinct domains and subsequently leveraging models to enumerate 1,600 applications. Based on these applications, we construct 80,000 mock tools, forming an extensive tool graph. Through random walk methodologies, we systematically sample subgraphs with predetermined node quantities from this comprehensive tool graph, and hence tool graph complexity is controlled via node quantity.
- **InstructionAgent** The difficulty of reasoning is quantified in the following dimensions: constraint complexity, quantity of reasoning points, and length of the reasoning chain. The model is required to generate instructions that comprehensively describe complete tasks based on the tool set extracted by the **ToolSetAgent**.
- **EnvironmentAgent** We augment environmental information including item details, location specifics, temporal parameters, and meteorological conditions based on content generated by the **UserProfileAgent** and **InstructionAgent**. Additionally, we introduce confounding elements for items and locations to further increase reasoning complexity.
- **RubricAgent** We construct a comprehensive series of specific checklists based on various task-related information. During final evaluation, considering the long-context characteristics inherent to agentic tasks, we employ a sliding window approach to assess the entire trajectory, continuously updating the completion status of checklist items.
- **ValidatorAgent** and **DeduplicatorAgent** We check the quality of our final tasks from several angles and remove any that are too similar. This process ensures we have a diverse and high-quality set of tasks.

With these high-quality challenging tasks, we further conduct rigorous response selection to construct our cold start training set with an appropriate quantity, revealing diverse patterns and preserving high exploration ability. We also carefully select a subset of these generated task for further post-training procedure, to make sure each task worth massive exploration.

4.3 General Capability

Instruction-following We curate both single-turn and multi-turn instruction-Following datasets, with varying levels of constraint complexity and quantity. For multiple-constraint queries, we adopt the insight from Ye et al. [2025] to filter queries with low semantic quality or constraint conflicts. For different query types, we employ verifiable rules, model-based verification, and customized strategies to ensure responses satisfy all constraints. Additionally, we compile critique datasets targeting challenging tasks to enhance the model’s critical thinking abilities [Wang et al., 2025c]. We observe that certain constraint types are inherently difficult to follow, making direct generation of valid query-answer pairs unreliable. To address this, we propose a reverse prompt generation strategy: generating queries from predefined answers guaranteed to meet constraints.

Long Context To enable the model to identify and analyze relevant information within complex, lengthy contexts, we develop three types of long-sequence datasets: reading comprehension, table-based question answering, and custom-designed tasks. To facilitate the learning of salient information in long sequences, we aggregate topically related context segments for data construction. We specifically enhance the model’s multi-hop reasoning, multi-turn dialogue, and complex calculation abilities. To mitigate hallucination when confronted with an incomplete context, we optimize the model’s refusal capabilities, thereby improving its awareness of knowledge boundaries and limitations.

Safety Building on the framework of Mu et al. [2024] and aligned with our internal content guidelines, we develop a content safety policy that categorizes queries into more than 40 distinct safety categories across five response types: *comply*, *comply with guideline*, *soft refuse*, *soft refuse with guideline*, or *hard refuse*. Explicit criteria ensure consistent, safety standards-compliant responses for each response type. This system operates as a context-aware data synthesizer through two stages: (1) Query Classification: Queries from diverse sources (open-domain corpora, internal business risk reports, government Q&A, and adversarial LLM-synthesized red-team content) are classified into safety categories using automated labeling with human verification. (2) Response Mapping & Optimization: Classified queries are mapped to response types and generate optimized, type-specific responses that undergo human evaluation before serving as training targets.

4.4 Evaluation

We conduct a comprehensive and rigorous evaluation of LongCat-Flash after post-training. Specifically, we assess its capabilities across multiple dimensions, including general domains, instruction following, mathematical reasoning, general reasoning, and coding & agent tasks.

4.4.1 Evaluation Benchmarks and Configurations

The evaluation employs the following benchmarks:

- **General Domains:** MMLU [Hendrycks et al., 2021a], MMLU-Pro [Wang et al., 2024b], ArenaHard [Li et al., 2024a,b], CEval [Huang et al., 2023], and CMMLU [Li et al., 2023a].
- **Instruction Following:** IFEval [Zhou et al., 2023], COLLIE [Yao et al., 2024], and Meeseeks [Wang et al., 2025a]. Meeseeks evaluates models’ instruction-following capabilities in multi-turn scenarios through an iterative feedback framework that simulates realistic human-LLM interactions, enabling models to self-correct based on turn-specific failures and better reflect real-world usage patterns.
- **Mathematical Reasoning:** MATH500 [Lightman et al., 2023], AIME24 [MAA, 2024], AIME25 [MAA, 2025], and BeyondAIME [ByteDance-Seed, 2025].
- **General Reasoning:** GPQA-diamond [Rein et al., 2023], DROP [Dua et al., 2019], ZebraLogic [Lin et al., 2025], and GraphWalks [OpenAI, 2025a].
- **Coding:** Humaneval+ [Liu et al., 2024b], MBPP+ [Liu et al., 2024b], LiveCodeBench (2024.08-2025.05) [Jain et al., 2025], SWE-Bench-Verified [Jimenez et al., 2024], and TerminalBench [Team, 2025a].
- **Agentic Tool Use:** τ^2 -Bench [Barres et al., 2025] and AceBench [Chen et al., 2025]. Furthermore, we develop a high-quality proprietary benchmark, VitaBench, leveraging Meituan’s comprehensive real-world business scenarios to systematically evaluate models’ capabilities in addressing complex real-world tasks. Within VitaBench, to comprehensively assess models’ generalized agentic capabilities, we deliberately curate cross-domain quotidian scenarios and explicitly delineate inter-tool dependencies, eschewing the provision of extensive domain-specific policies. Our benchmark emphasizes three critical dimensions of complexity: tool set complexity (characterized by dense tool graphs averaging over 30 available tools per task), reasoning complexity, and user interaction complexity (featuring challenging user personas with an average exceeding 60 interaction rounds per task for evaluated models). The complete benchmark dataset, along with detailed construction methodologies and comprehensive result analysis, will be fully released in subsequent work.

We also evaluate the safety performance of LongCat-Flash. Specifically, we conduct evaluations on four major risk categories:

- **Harmful:** Violence, hate Speech, insulting, harassment and bullying, self-harm and suicide, adult content, etc.
- **Criminal:** Illegal activities, underage violations, extreme terrorism and violence, etc.
- **Misinformation:** misinformation and disinformation, unsafe practices, hallucination, etc.
- **Privacy:** privacy violation, infringement, etc.

Within each category, a sufficient number of private test queries are constructed, followed by a comprehensive manual review to ensure the accuracy of their classification and the reliability of their quality.

We compare the chat version of LongCat-Flash with several contemporary non-thinking chat models, including DeepSeek-V3.1 [DeepSeek-AI et al., 2025], Qwen3-235B-A22B (2507 version) [Yang et al., 2025], Kimi-K2 [MoonshotAI, 2025], GPT-4.1 [OpenAI, 2025b], Claude4-Sonnet [Anthropic, 2025], and Gemini2.5-Flash [Comanici et al., 2025]. For closed-source models, we conduct evaluations through their official APIs. For models supporting both thinking and non-thinking modes (Qwen3-235B-A22B, Gemini2.5-Flash, and Claude4-Sonnet), we explicitly configure these models to operate in non-thinking mode for a fair comparison.

For each benchmark category, we employ the following specialized metrics and settings:

- **General domain benchmarks:** We use accuracy as the evaluation metric. Unlike the original benchmarks that rely on exact-match (EM) for correctness judgment, we employ a scoring model to assess whether model responses align with reference answers. Since our scoring model recognizes semantically correct answers even without exact textual matches, reported values may be slightly higher than originally documented.
- **Instruction following benchmarks:** We design regular expressions based on instruction rules to verify compliance. Rule-based and model-based answer span extraction tools are additionally employed to support this evaluation.

- **Mathematical reasoning benchmarks:** We apply the aforementioned scoring model for MATH500, and the averaged EM scores over 10 runs for AIME-related benchmarks.
- **General reasoning benchmarks:** We apply the scoring model for GPQA-diamond, calculate the F1 score for DROP, adopt rule-based matching for ZebraLogic, and use the precision metric for GraphWalk following the official implementation on its 128k context length subset.
- **Coding benchmarks:** Each problem is scored 1 if the model’s response passes all test cases in a sandbox environment or matches a specific state, otherwise 0. The final score is the average across all problems. We adopt the script provided by OpenAI² to evaluate Humaneval+ and MBPP+, and the official scripts for the others. Specifically, for SWE-Bench-Verified, we use R2E-Gym³ (Openhands scaffold) with runs limited to 100 iterations for evaluation except DeepSeek V3.1 (using Openhands⁴). For Terminal-Bench, we use the Terminus framework with direct prompting for evaluation.
- **Agentic tool use benchmarks:** We utilize official benchmark frameworks to ensure fairness and reproducibility. For AceBench, we use direct prompting rather than function calling. For our proposed **VitaBench**, given the inherent long-context characteristics of agentic tasks, we employ a sliding window mechanism to systematically evaluate task completion status throughout the entire execution trajectory, facilitating continuous updates to the completion status of individual checklist components.

4.4.2 Evaluation Results

As detailed in Table 3, our comprehensive evaluation reveals that LongCat-Flash is a powerful and versatile model. It consistently demonstrates leading performance in different domains, often outperforming contemporary models across a wide array of challenging tasks with relatively fewer activated parameters. The following analysis provides a detailed breakdown of its impressive capabilities across different dimensions.

General Domains In general domain knowledge, LongCat-Flash demonstrates a strong and well-rounded performance. It achieves an excellent score of 86.50 on ArenaHard-V2, ranking second among all evaluated models and showcasing its robust capabilities in challenging head-to-head comparisons. On foundational benchmarks, it remains highly competitive, scoring 89.71 on MMLU and 90.44 on CEval. These results are comparable to leading models, and notably, are achieved with fewer parameters than competitors like DeepSeek-V3.1 and Kimi-K2, indicating high efficiency.

Instruction Following LongCat-Flash exhibits state-of-the-art instruction following capabilities. It achieves the highest score of 89.65 on IFEval, outperforming all other models and demonstrating superior reliability in adhering to complex and nuanced directives. Furthermore, it secures the best score on COLLIE (57.10) and Meeseeks-zh (43.03), underscoring its exceptional proficiency across diverse and challenging instruction sets in both English and Chinese.

Mathematical Reasoning In mathematical reasoning, LongCat-Flash shows powerful and advanced capabilities. While its score on MATH500 (96.40) is highly competent, its strength is particularly evident in more complex, competition-level benchmarks. It delivers excellent, top-tier results on AIME25 (61.25) and BeyondAIME (43.00), ranking among the best-performing models in these challenging domains. This highlights its advanced capacity for sophisticated, multi-step logical deduction and problem-solving.

General Reasoning For general reasoning tasks, LongCat-Flash’s performance is also solid. It demonstrates exceptional strength in structured logical deduction, achieving a score of 89.30 on ZebraLogic, which is among the top competitors. It also obtains a competitive score of 79.06 on the reading comprehension benchmark DROP. Conversely, its results on GPQA-diamond (73.23) and GraphWalks (51.05) indicate an opportunity for further improvement, particularly in enhancing its capabilities for analyzing structured data within extremely long contexts.

Coding LongCat-Flash displays a promising and capable profile in the coding domain. Its standout performance is on TerminalBench, where it achieves a score of 39.51, ranking second and demonstrating excellent proficiency in practical, agentic command-line tasks. It is also competitive on the SWE-Bench-Verified benchmark with a score of 60.4. On foundational code generation tasks such as Humaneval+ and MBPP+, its performance is solid, yet there remains potential for future optimization to align with the leading models.

Agentic Tool Use LongCat-Flash demonstrates a clear advantage in using agentic tool use domain, notably outperforming other models on τ^2 -Bench even when compared to models with more parameters. In highly complex scenarios, it achieves the highest score of 24.30 on VitaBench, demonstrated strong capability in complex scenarios.

²<https://github.com/bigcode-project/bigcode-evaluation-harness>

³<https://github.com/R2E-Gym/R2E-Gym>

⁴<https://github.com/All-Hands-AI/OpenHands>

Safety LongCat-Flash showed outstanding capability in identifying and mitigating risks on the whole, particularly in the domains of Harmful and Criminal compared to other models.

Table 3: Evaluation results of frontier chat models. Values marked with * are sourced from other public reports. Note that DeepSeek-V3.1, Qwen3-235B-A22B, Gemini2.5-Flash, and Claude4-Sonnet are evaluated under their non-thinking mode.

Benchmark	DeepSeek V3.1	Qwen3 MoE-2507	Kimi-K2	GPT-4.1	Claude4 Sonnet	Gemini2.5 Flash	LongCat-Flash
Architecture	MoE	MoE	MoE	-	-	-	MoE
# Total Params	671B	235B	1043B	-	-	-	560B
# Activated Params	37B	22B	32B	-	-	-	27B
General Domains							
MMLU _(acc)	90.96	90.23	89.86	89.64	91.75	86.33	89.71
MMLU-Pro _(acc)	84.45	84.83	82.06	81.72	83.74	81.95	82.68
ArenaHard-V2 _(acc)	84.10	88.20	85.70	61.50	62.10	77.00	86.50
CEval _(acc)	89.21	92.70	91.26	79.53	86.63	78.78	90.44
CMMLU _(acc)	88.04	88.14	89.66	77.65	86.51	78.30	84.34
Instruction Following							
IFEval _(acc)	86.69	88.54	88.91	85.58	88.35	83.92	89.65
COLLIE _(acc)	43.80	49.71	56.34	50.00	51.22	48.60	57.10
Meeseeks-zh _(acc)	33.83	35.32	42.79	41.54	35.07	34.84	43.03
Mathematical Reasoning							
MATH500 _(acc)	96.08	98.80	97.60	90.60	93.80	98.40	96.40
AIME24 _(avg@10)	66.30*	81.67	69.60*	47.00	47.00	79.67	70.42
AIME25 _(avg@10)	49.27	68.33	50.66	32.00	37.00	67.33	61.25
BeyondAIME _(avg@10)	36.50	57.60	36.60	22.10	20.50	44.20	43.00
General Reasoning							
GPQA-diamond _(acc)	74.90*	77.43	75.76	67.68	70.71	80.30	73.23
DROP _(f1)	84.19	78.57	89.04	66.94	73.06	45.03	79.06
ZebraLogic _(acc)	85.30	94.22	89.11	56.30*	75.85	51.78	89.30
GraphWalks-128k _(precision)	73.54	80.72	47.50	85.02	80.57	64.83	51.05
Coding							
LiveCodeBench _(pass@1)	56.40*	46.48	46.70	39.21	45.59	39.65	48.02
Humaneval+ _(pass@1)	92.68	94.51	85.98	93.29	94.51	87.80	88.41
MBPP+ _(pass@1)	79.89	79.89	81.75	79.37	80.16	76.19	79.63
SWE-Bench-Verified _(acc)	66.00*	42.00	64.60	48.60	68.00*	40.60	60.40
TerminalBench _(acc)	31.30*	17.28	25.93	28.40	40.74	12.35	39.51
Agentic Tool Use							
τ^2 -Bench (telecom) _(avg@4)	38.50	22.50	67.50	35.20	46.20	16.50	73.68
τ^2 -Bench (airline) _(avg@4)	46.00	36.00	54.20	56.00	60.00	41.50	58.00
τ^2 -Bench (retail) _(avg@4)	64.90	70.50	70.80	74.10	80.00	64.80	71.27
AceBench _(acc)	69.70	71.10	82.20	80.10*	76.20*	74.50*	76.10
VitaBench _(avg@4)	20.30	8.50	18.20	19.00	23.00	8.00	24.30
Safety							
Harmful	82.79	80.82	53.91	56.19	66.56	-	83.98
Criminal	87.83	89.13	77.19	81.58	87.58	-	91.24
Misinformation	83.17	77.76	42.68	45.49	54.91	-	81.72
Privacy	98.80	98.80	96.39	98.80	100.00	-	93.98

5 Training Infrastructures

The core design principle of our training infrastructure is scalability with precision. We developed a systematic method to verify operator precision and embedded online Silent Data Corruption (SDC) detection into idle computation phases to minimize numerical errors. To guarantee reproducibility and ensure consistent results between small-scale

experiments and full-scale training, we enforced determinism across all computation and communication operators. This enabled bitwise-aligned loss values across multiple re-runs of any training step.

With correctness ensured, we focused on accelerating training efficiency. Wall-clock time is critical for rapid algorithm iteration, yet single accelerator provides limited capability. We therefore scaled training across tens of thousands of accelerators, confronting challenges in scalability and stability. Through model–system co-design, multi-dimensional parallelism, and fully automated fault detection and recovery, we achieved near-linear scaling and 98.48% availability, completing training within 30 days.

5.1 Numerical Precision Control and Fault Detection

ULP Evaluation Floating-point errors are influenced by multiple factors, even varying between accelerators of the same vendor across generations. To quantify and mitigate these errors, we adopt ULP (Unit in the Last Place) as a metric, where ULP error measures the deviation of accelerator BF16 results from CPU FP32 ground truth. A zero ULP error indicates perfect accuracy, while larger values imply worse precision. We collect all operator types and shapes used in training and compare their ULP errors. Table 4 shows the ULP error for GEMM between two solutions.

Table 4: GEMM Precision Comparison (ULP)

Case	Output Shape	Value Range	Solution 1		Solution 2	
			Max	Min	Max	Min
1	[1024,1536]	[-5,5]	2292	-568	112	-100
2	[1024,576]	[-5,5]	65362	-82046	6.5	-9
3	[1024,16384]	[-19,15]	544	-104	224	-112
4	[1024,12288]	[-4,4]	202	-88	72	-41
5	[1024,6144]	[-1,1]	5376	-1376	304	-224
6	[1024,24576]	[-5,5]	7200	-510	104	-294
7	[1024,131072]	[5,5]	8128	-6976	2528	-368
8	[1024,6144]	[-1,1]	5344	-8064	80	-258

SDC Detection Mechanism SDC faults are typically unavoidable in large-scale training and can severely degrade model performance by altering data without system warnings. To address this, we implement an efficient on-chip in-place operator recomputation mechanism. Specifically, we find that the backward computation for FlashAttention Gradients (FAG) is most sensitive to SDC because it simultaneously mixes tensor and vector computations. Bitwise differences between recomputed results indicate potential SDC risks. The detection computations are orchestrated within compute streams, and the recomputation interval is manually adjustable, enabling a flexible trade-off between detection coverage and computational cost.

Notably, operator precision control is necessary but insufficient for ensuring model accuracy. Experiments with different operator implementations may show training loss discrepancies within $1e-3 \sim 1e-4$ yet exhibit larger than 5 pp variation on benchmarks. Cost-effectively evaluating the impact of operator precision errors on model performance remains an open challenge.

5.2 Kernel Optimization for Determinism and Performance

Determinism serves as the gold standard for computational correctness, eliminating floating-point errors as experimental variables. However, achieving determinism often incurs significant performance overhead. We address this through kernel redesigns, maintaining deterministic computation and communication throughout LongCat-Flash’s training.

Deterministic FAG The default FAG implementation is non-deterministic because dQ , dK , and dV are reduced along different dimensions, where atomic addition lacks order preservation. We develop an efficient deterministic FAG kernel using limited extra workspace to accumulate tiles in a deterministic order. With co-optimizations including double-buffer pipelining, tuned tiling schedules, and load balancing, our implementation achieves 1.6x the performance of the original deterministic version and 0.95x that of the non-deterministic version, striking a balance between determinism and efficiency.

Deterministic ScatterAdd ScatterAdd in backward passes is essential for gradient aggregation but suffers from input-output operand counts mismatches. The default implementation enforces sequential execution within a single compute unit, causing up to 50x slowdown. We propose a hierarchical reduction algorithm that parallelizes gradient aggregation across all available processors, achieving performance parity with the non-deterministic version.

Optimized Grouped GEMM Grouped GEMM’s performance is critical given its high computational volume but low compute density versus dense GEMM. We optimize it via: (1) Double-buffer pipelining to overlap computation, memory I/O, and epilogue; (2) Diagonal tiling to mitigate L2 cache conflicts; (3) HBM bandwidth control via compute unit limits to overlap Grouped GEMM with dispatch/combine communication. These optimizations yield 5%–45% speedups over the default version.

Fused GemmAdd The *dw* computation suffers bandwidth-bound bottlenecks during gradient accumulation. We fuse FP32 addition into the GEMM epilogue, avoiding intermediate write-backs and hiding addition within tile GEMM pipelines. This significantly reduces latency and eliminates the precision loss caused by the conversion of BF16 data to HBM, achieving a speedup of 3.12x to 3.86x on the fused GroupedGemmAdd benchmark.

Furthermore, we re-implement IO-bound kernels (e.g., MoE layer permute/unpermute) with integrated functionalities like drop-token and zero-computation experts handling, ensuring both determinism and performance.

5.3 Distributed Strategy for Large-scale Training

The training architecture is centered on Expert Parallelism Groups (EP), each comprising 32 accelerators. Within an EP Group, the attention layer employs Context Parallelism (CP=8) instead of Tensor Parallelism (TP) to minimize communication overhead, and the FFN layer uses EP partitioning without TP. Multiple EP groups are scaled across Pipeline Parallelism (PP) and Data Parallelism (DP) dimensions.

Expert parallelism (EP) is adopted to reduce static memory usage, including weights and optimizer states. However, EP inherently introduces costly dispatch and combine communication operations. To mitigate this, LongCat-Flash adopts the ScMoE structure, which enables dispatch/combine communication to overlap by more computation in a single batch. Furthermore, the MoE layer is divided into two chunks along the token dimension. These subchunks achieve two objectives: (1) Overlap with the dense FFN computation. (2) Overlap with each other (see Figure 8).

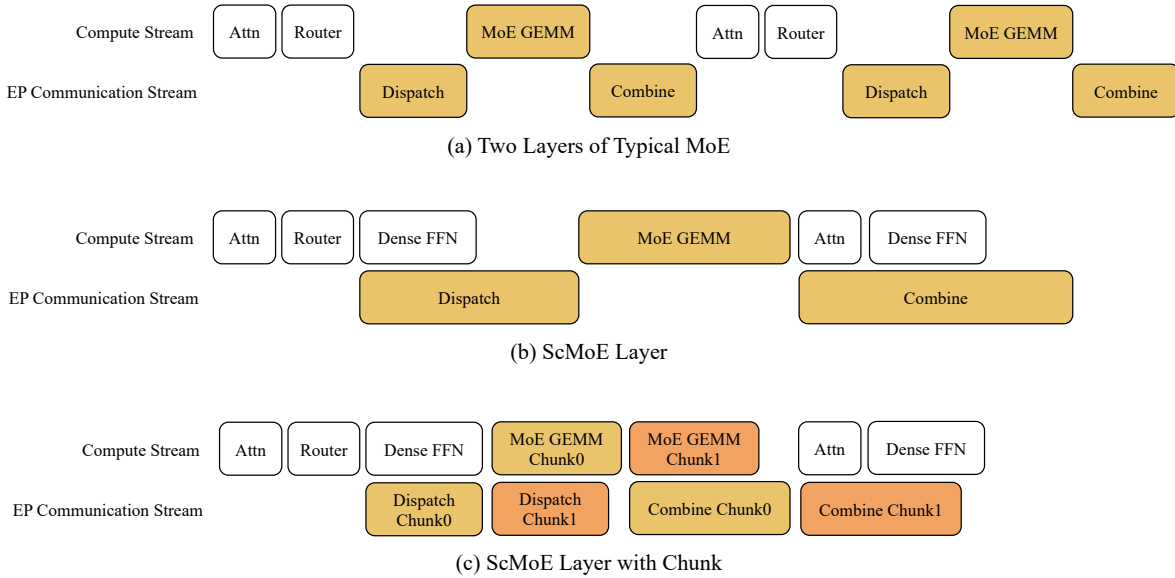


Figure 8: These architectures have the same total and activated number of experts. ScMoE with chunk achieves the highest efficiency because more communication is overlapped by computation.

There are two optimized strategies for dispatch/combine communication: (1) All-gather/reduce-scatter kernel with pipeline in the intranode and the internode; (2) Optimized all-to-all kernel. The native all-to-all expands the local data size by top-k times, increasing traffic through the 200Gb/s per accelerator RDMA network. Additionally, all-to-all performance is unstable due to inadequate congestion control. We select pipelined all-gather/reduce-scatter with deterministic as the primary solution, the proportion of time to non-overlapping dispatch/combine communication decreases from 25.3% to 8.4% with ScMoE architecture.

Existing pipeline strategies (e.g., 1F1B, interleaved-1F1B, Zero-bubble [Qi and Others, 2023]) suffer from imbalanced memory usage across pipeline stages. To this end, we adopt the V-ZB algorithm [Qi et al., 2024], which balances

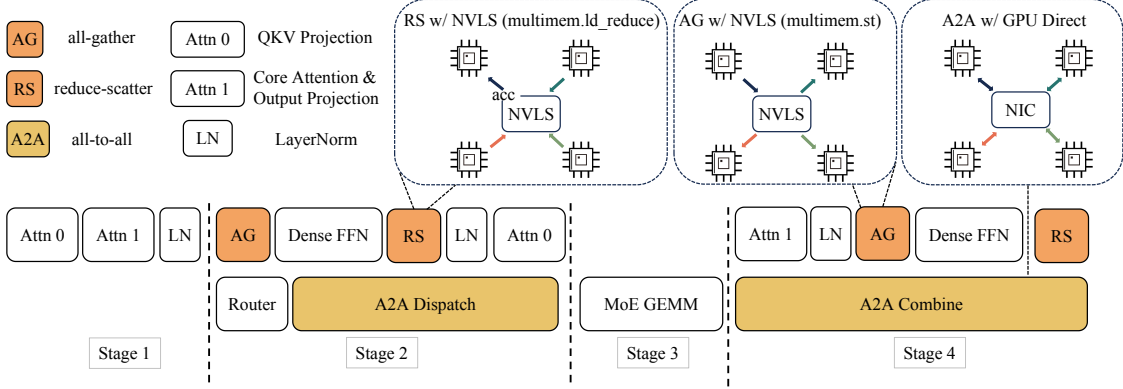


Figure 9: An overview of overlapping strategy.

memory usage at all stages and reduces peak memory to less than 60GB in the training of LongCat-Flash. Additionally, we enable the post-validation strategy from zero bubble, achieving zero theoretical bubbles. A key refinement is replacing inverse operations with backup data from the previous step during optimizer state rollback, preserving numerical bitwise alignment.

5.4 Reliability and Observability

Reliability is measured by the proportion of time contributing to the final training trajectory (Availability), where unavailable time includes fault recovery and wasted time between the last checkpoint and fault occurrence. Asynchronous checkpointing reduces training stall to 2~4 seconds, allowing higher frequency and minimizing fault-induced loss. Combined with online critical log filtering, optimized initialization, and full automation, recovery time is reduced to <10 minutes. These mechanisms achieve 98.48% availability, with all 20 faults handled automatically without manual intervention.

Observability combines fine- and coarse-grained profiling with a metric platform. Fine-grained PyTorch profiler timelines enable distributed, parallel-aware co-analysis to identify pipeline parallelism "bubbles" and inter-rank communication waits. Coarse-grained monitoring adds low-overhead runtime analysis of stragglers. The metric platform tracks loss, weights, gradients, and activations for rapid model state assessment.

6 Inference and Deployment

LongCat-Flash employs a model-system co-design, which significantly contributes to its high throughput and low latency. This section focuses on inference optimizations implemented in one of our deployment clusters, presenting methods to simultaneously boost system throughput and significantly reduce latency to 100 TPS on H800. We first present our parallel inference architecture co-designed with the model architecture. Following the inference architecture, optimization methods such as quantization and custom kernel are described. Finally, we present our deployment strategy and performance results.

6.1 Model-Specific Inference Optimization

To achieve an efficient inference system, two key challenges must be addressed: (1) Computation and communication orchestration, and (2) KV cache I/O and storage. For the first challenge, existing approaches typically exploit parallelism at three conventional granularities: operator-level overlap like NanoFlow [Zhu et al., 2025], expert-level overlap represented by EPS-MoE [Qian et al., 2025], and layer-level overlap demonstrated in DeepSeek-V3 TBO (Two Batch Overlap) [Team, 2025b]. LongCat-Flash’s ScMoE architecture introduces a fourth dimension—module-level overlap—for which we designed the SBO (Single Batch Overlap) scheduling strategy to optimize both latency and throughput. For the second challenge—KV cache I/O and storage—LongCat-Flash addresses these issues through architectural innovations in its attention mechanism and MTP structure to reduce the effective I/O overhead.

6.1.1 Computation and Communication Orchestration

LongCat-Flash naturally exhibits computation-communication overlap properties in its structure, which is the key to achieving lower latency while maintaining generation throughput. We carefully design Single Batch Overlap (SBO), a four-stage pipeline execution that uses module-level overlap to fully unleash LongCat-Flash’s potential as shown in Figure 9. SBO differs from TBO by hiding communication overhead within a single batch. In SBO, stage 1 requires separate execution because the MLA output serves as input for subsequent stages. In stage 2, we overlap all-to-all dispatch with Dense FFN and Attn 0 (QKV Projection). This overlap is crucial because communication overhead is excessive, prompting us to split the attention process. Stage 3 independently executes MoE GEMM. The latency of this stage will benefit from the wide EP deployment strategy. In stage 4, we overlap Attn 1 (Core Attention and Output Projection) and Dense FFN with the all-to-all combine. This orchestration effectively mitigates the communication overhead, ensuring efficient inference for LongCat-Flash.

Additionally, the ScMoE architecture, under the wide EP deployment scheme, facilitates the overlap of intra-node NVLink bandwidth utilization and inter-node RDMA communication through GPUDirect RDMA [Choquette, 2022], thereby improving overall bandwidth efficiency. Dense FFN in ScMoE has a relatively large intermediate size, so TP deployment is employed to minimize memory footprint, necessitating all-gather and reduce-scatter communication before and after Dense FFN, respectively. To reduce this communication overhead, we develop custom kernels and adopt TP2 or TP4 instead of TP8.

6.1.2 Speculative Decoding

LongCat-Flash employs MTP as the draft model for speculative decoding. Our optimization framework originates from a systematic breakdown of Speculative Decoding’s speedup formulation, as Sadhukhan et al. [2025] has mentioned:

$$\frac{T_{Avg}^{SD}}{T_T} = \frac{1}{\Omega(\gamma, \alpha)} \left(\frac{\gamma \cdot T_D}{T_T} + \frac{T_V(\gamma)}{T_T} \right),$$

where T_{Avg}^{SD} , T_T , T_D are expected latency per token for speculative decoding, target model and draft model. γ represents number of draft token in one decoding step. $\Omega(\gamma, \alpha)$ is expected accept length for a given step γ and acceptance rate α . And $T_V(\gamma)$ is expected latency for target verification. Our approach targets three key factors:

- Expected accept length $\Omega(\gamma, \alpha)$, which is positively correlated with the acceptance rate α of draft tokens. To maximize acceptance rate α , we employ MTP. Integrate a single MTP head during late-phase pre-training, achieving approximately 90% acceptance rate on test sets.
- Draft to target cost ratio $\gamma \frac{T_D}{T_T}$, which is dominated by the structures of both target model and draft model. As noted by Liu et al. [2024c], balancing draft quality and speed is critical. To minimize generation overhead while maintaining comparable acceptance rates, LongCat-Flash adopts a lightweight MTP architecture with reduced parameters. Our experiments (Table 5) show that a single dense layer for MTP heads optimizes this trade-off, outperforming ScMoE layers in latency.
- Target verification to decoding cost ratio $\frac{T_V(\gamma)}{T_T}$. In order to reduce this ratio, we adopt the C2T [Huo et al., 2025] method, using a classification model to filter out tokens that are unlikely to be accepted before verification.

Table 5: Draft token acceptance rate on MT-Bench of different MTP head structures with a 6B activated model. The ratio of MTP head parameters to main model parameters is also reported.

MTP layer	Activated parameters ratio	Acceptance rate α
Dense layer	1.41%	92.1%
ScMoE layer	4.17%	92.9%

6.1.3 Reducing KV Cache

To balance performance and efficiency, LongCat-Flash adopts MLA with 64 heads for its attention mechanism, which reduces the computational load of the attention component while achieves exceptional KV cache compression and thus reduces storage and bandwidth pressure. This is crucial for orchestrating LongCat-Flash’s pipeline, as noted in Figure 9 the model always features an attention computation that cannot be overlapped with communication. Specifically, the MQA-like structure of the MLA absorb method shares KV across the m-dimension (64 heads), aligning with the shape of the WGMMMA instruction for maximal hardware utilization.

6.2 System-Wide Inference Techniques

6.2.1 Minimize Schedule Overhead

The decoding phase in LLM inference systems can become launch-bound due to kernel launch overhead. This issue is exacerbated when introducing speculative decoding—particularly with LongCat-Flash’s lightweight MTP, where separate scheduling of verification kernels and draft forward passes introduces significant overhead. To mitigate this, a TVD fusing strategy is used to fuse Target forward, Verification, and Draft forward into a single CUDA graph. To further improve GPU utilization, we implement an overlapped scheduler. However, experimental results reveal that the low latency of LongCat-Flash’s forward pass renders a single-step pre-schedule strategy insufficient to fully eliminate scheduling overhead. As shown in Figure 10, we introduce a multi-step overlapped scheduler to launch the kernel for multiple forward steps in a single schedule iteration. This approach effectively hides CPU scheduling and synchronization within the GPU forward process, ensuring continuous GPU occupancy.

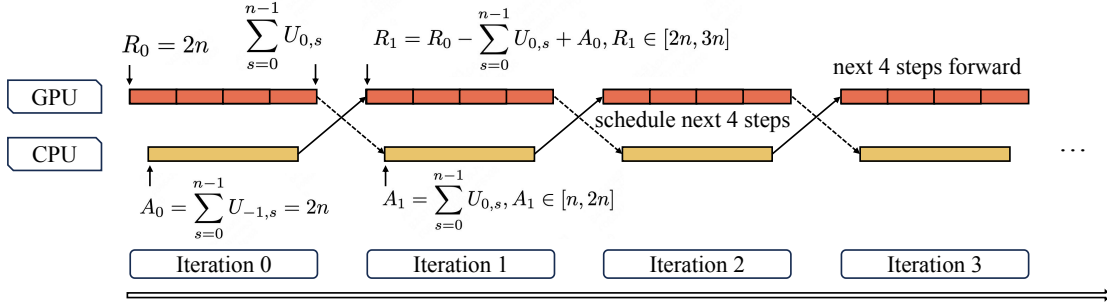


Figure 10: Multi-step overlapped scheduler (4 steps as an example here).

In a multi-step overlapped scheduler, we need to dynamically pre-allocate KV cache slots for multiple future steps without prior knowledge of the accept length of speculative decoding in previous iterations. An important issue is whether multi-step overlapped scheduling causes divergent KV cache allocation. We illustrate this with $MTP = 1$ and the number of steps, $n = 4$. Let R_i represents available KV entries during the GPU’s i -th iteration forward pass, thus $R_0 = (MTP + 1) \times n = 2n$. $U_{i,s} \in [1, 2]$ represents the accept length in the i -th iteration for the s step, with the initial value $U_{-1,s} = 2$. Then, while the GPU is performing the i -th iteration of forward computation, the scheduler pre-allocates the KV cache slots needed for the $(i + 1)$ -th forward iteration based on the accept length in the $(i - 1)$ -th forward iteration, where A_i represents the KV cache slots that is allocated. Formally:

$$A_i = \sum_{s=0}^{n-1} U_{i-1,s}, \quad i \geq 0$$

$$R_i = R_{i-1} - \sum_{s=0}^{n-1} U_{i-1,s} + A_{i-1}, \quad i \geq 1$$

By induction, we obtain the closed-form expression:

$$R_i = 4n - \sum_{s=0}^{n-1} U_{i-1,s}, \quad i \geq 1$$

which means:

$$R_i \in [2n, 3n], \quad i \geq 1$$

Through mathematical induction, this ensures safe KV cache allocation for the next iteration even without knowing the current iteration’s accept length, while guaranteeing convergence in allocated KV cache size.

6.2.2 Custom Kernel

The autoregressive nature of LLM inference creates distinct efficiency challenges. The prefilling phase is compute-bound, and methods like chunk-prefill [Agrawal et al., 2023] regularize data for optimal processing. In contrast, the

decoding phase is often memory-bound due to small, irregular batch sizes from traffic patterns, which hurts kernel performance. Therefore, optimizing these specific cases is crucial for minimizing Time-Per-Output-Token (TPOT).

MoE GEMM Existing libraries like DeepGEMM [Zhao et al., 2025a] map model weights to right-hand matrices (B in $A \times B = C$) aligned with k/n dimensions, while input activations become left-hand matrices mapped to m/k dimensions, where m represents token count. This conventional approach requires padding when token counts fall below m 's 64-element minimum. To address this inefficiency, we leverage the SwapAB [Dege et al., 2025] technique: treating weights as left-hand matrices and activations as right-hand matrices. By exploiting the n -dimension's flexible 8-element granularity, SwapAB maximizes tensor core utilization.

Communication Kernels The inference system leverages NVLink Sharp's hardware-accelerated broadcast (multimem.st) and in-switch reduction (multimem.ld_reduce) to minimize data movement and SM occupancy, as shown in Figure 9. By using inline PTX assembly, the reduce-scatter and all-gather kernels enable high-efficiency data transmission. These kernels support both uniform and nonuniform token distributions across GPUs, and consistently outperform NCCL [NVIDIA] and MSCCL++ [Shah et al., 2025] across 4KB to 96MB message sizes, using only 4 thread blocks.

6.2.3 Quantization

LongCat-Flash employs the same quantization scheme as DeepSeek-V3, using fine-grained block-wise quantization: activations per $[1, 128]$ blocks and weights per $[128, 128]$ blocks. Besides, to achieve an optimal performance-accuracy trade-off, we applied layer-wise mixed-precision quantization based on two methodologies: The first scheme follows our approaches in FPTQ [Li et al., 2023b] and Super-Expert [Su et al., 2025], where we observed that certain linear layers (particularly Downproj) exhibited input activations with extreme magnitudes reaching 10^6 . The second scheme involves computing block-wise FP8 quantization errors (both relative and absolute) layer by layer, which revealed significant quantization errors in specific expert layers. By taking the intersection of both schemes, we achieved substantial accuracy improvements.

6.3 Deployment and Performance

6.3.1 Measured Performance

Table 6: Performance of LongCat-Flash under different settings.

Model	Attention	Avg Context	#Hopper GPUs	TGS	TPS/u
DeepSeek-V3-profile	bf16	4096	128	2324	20
DeepSeek-V3-blog	bf16	4989	144	1850	20 ~22
LongCat-Flash	bf16	5000	128	3785	35
LongCat-Flash	bf16	5000	128	2205	68.9
LongCat-Flash	bf16	5000	128	804	100.5
LongCat-Flash	fp8	5000	128	4230	26.4
LongCat-Flash	fp8	8192	128	3240	33.8

To enable independent optimization of prefilling and decoding phases, PD-Disaggregated architecture is adopted. A key challenge in this design is the overhead of transmitting KV caches from prefilling to decoding nodes. To mitigate this, we implement layer-wise transmission, which significantly reduces Time-To-First-Token (TTFT) under high QPS workloads. For prefilling and decoding nodes, the minimum deployment unit consists of 2 nodes with 16 H800-80GB GPUs. Meanwhile, wide EP is deployed with DeepEP [Zhao et al., 2025b] to minimize communication overhead. Besides, we modify DeepEP and EPLB (Expert Parallelism Load Balancer) to support zero-computation experts, where the outputs of zero-computation experts can be obtained without communication.

Table 6 compares the throughput and latency of LongCat-Flash with DeepSeek-V3 (DeepSeek-V3-profile from DeepSeek [2025a], DeepSeek-V3-blog from DeepSeek [2025b]), where TGS (token per GPU per second) represents generation throughput per device (higher values indicate lower cost), and TPS/u (tokens per second per user) represents the generation speed for one user (higher values are better). During testing, the steady-state generation throughput under a given sequence length is used for calculation. LongCat-Flash achieves higher generation throughput and faster generation speed across different sequence lengths.

In Agent applications based on the ReACT [Yao et al., 2023] pattern, completing a single task requires multiple rounds of model interactions, where interaction latency directly impacts user experience. Analysis of typical Agent invocation patterns reveals differentiated speed requirements for model outputs:

- Reasoning content (user-visible): consisting of cognitive processes and explanations, must match human reading speed (20 tokens/s).
- Action commands (user-invisible): structured data such as function names and parameters, typically 30~100 tokens, yet directly affect tool invocation startup time—demanding the highest possible speed.

To address this scenario, LongCat-Flash achieves a generation speed of nearly 100 tokens/s for action commands. Under a cost assumption of \$2 per hour for an H800 GPU, this translates to a price of \$0.7 per million output tokens. This performance constrains the single-round tool-call latency to under one second, thereby significantly enhancing the interactivity of Agent applications.

6.3.2 Theoretical Performance

Figure 9 shows that LongCat-Flash’s latency is primarily determined by three components:

- MLA: Its time consumption cannot be reduced by increasing the number of EP.
- All-to-all dispatch/combine: Both are constrained by single-device batch size and topk.
- MoE: Its time consumption in the memory-bound region decreases with increasing EP count.

Assuming the number of EP is 128, MLA uses DP for DeepSeek-V3 and LongCat-Flash, GQA uses TP4 for Qwen3-235B-A22B as it has 4 kv heads, and the batch size per device is 96. Actually, the GQA feature of Qwen-235B-A22B results in a relatively high memory footprint for its KV cache, making it difficult to achieve a per-GPU batch size of 96 in practice. The assumption that it can reach this value is made here solely for the purpose of theoretical analysis. As pointed out by [Jiashi Li, 2025], FlashMLA can achieve up to 660 TFlops on NVIDIA H800 SXM5 GPUs; Zhao et al. [2025b] indicates that DeepEP bandwidth can reach 40GB/s. Both of these metrics were utilized in our computations. Assuming the cost for per H800 is \$2 per hour. Considering MTP=1 with an acceptance rate of 80%, we can calculate the theoretical time consumption and cost of each module in one layer of DeepSeek-V3, Qwen3-235B-A22B and LongCat-Flash, as listed in Table 7. For Qwen3-235B-A22B, which does not natively support MTP, we assume a speculative sampling strategy with a comparable acceptance rate.

Table 7: Theoretical decoding time and cost of different models.

	DeepSeek-V3	Qwen3-235B-A22B	LongCat-Flash
MTP	w/	w/o	w/
n_layer	61	94	28
batch per device	96	96	96
Time cost of different modules in one layer			
attention	471 us	314 us	264 us
all-to-all dispatch	275 us	157 us	236 us
MoE	77 us	29 us	60 us
all-to-all combine	551 us	315 us	472 us
TPOT and Price			
overlap strategy	TBO	TBO	SBO
TPOT (ms)	30	26.2	16
\$/1M output token	0.17	0.15	0.09

Under this configuration, the theoretical extreme TPOT for LongCat-Flash with SBO can be expressed as:

$$\begin{aligned} \text{TPL} &= 264 + 236 + 60 + 472 = 1032 \text{ us}, \\ \text{TPOT} &= \frac{28 \times \text{TPL}}{1000 \times 1.8} \approx 16 \text{ ms}, \end{aligned}$$

where TPL denotes the time cost per layer.

The measured value under batch size 96 is approximately TPOT = 26 ms, which is about 61.5% of the theoretical value and is on par with DeepSeek-V3 (~64%). The gap between measured value and theoretical speed mainly comes from the overhead of small operators and the loss in communication bandwidth.

We apply the same method to calculate the theoretical limits of TPOT and generation cost for DeepSeek-V3 and Qwen3-235B-A22B under TBO scheduling. It can be observed from Table 7 that through model system co-design, LongCat-Flash achieves significant theoretical improvements in both throughput and latency.

Furthermore, we observed two key insights about LongCat-Flash: (1) LongCat-Flash exposes not only all-to-all communication and MoE computation, but also an MLA computation. As a result, at the same batch size, LongCat-Flash incurs slightly longer per-layer time than DeepSeek-V3. However, due to its significantly reduced layer count, LongCat-Flash achieves lower overall latency. (2) LongCat-Flash’s second MLA is overlapped by the all-to-all combine. This means that in the decoding phase, LongCat-Flash can increase the sequence length to a certain extent without substantial latency increase.

7 Conclusion

We introduce LongCat-Flash, a 560B-parameter MoE model featuring three key innovations: (1) a context-aware dynamical computation mechanism and shortcut-connection MoE, enabling high efficiency in both training and inference, (2) integrated strategies that ensure stable large-scale training, (3) a multi-stage training pipeline that cultivates LongCat-Flash’s agentic capabilities, allowing it to perform complex tasks requiring iterative reasoning and environmental interaction. By releasing LongCat-Flash as an open-source model, we aim to advance research in efficient MoE architectures, high-quality data strategies, and agentic model development, fostering community-driven innovation in large language models.

8 Contributions

The listing of authors is in alphabetical order. Names marked with an asterisk (*) indicate people who have left our team.

Bayan	Jiahuan Li	Qiyuan Duan	Xuemiao Zhang
Bei Li	Jiajun Yang	Ran Meng	Xueyuan Hao
Bingye Lei	Jiaming Wang	Rongxiang Weng	Xuezhi Cao
Bo Wang	Jian Yang	Ruichen Shao	Xunliang Cai
Bolin Rong	Jianchao Tan	Rumei Li	Xurui Yang
Chao Wang	Jiaqi Sun	Shizhe Wu	Yan Feng
Chao Zhang	Jiaqi Zhang	Shuai Liang	Yang Bai
Chen Gao	Jiawei Fu	Shuo Wang	Yang Chen
Chen Zhang	Jiawei Yang	Suogui Dang	Yang Yang
Cheng Sun	Jiaxi Hu	Tao Fang	Yaqi Huo
Chengcheng Han	Jiayu Qin	Tao Li	Yerui Sun
Chenguang Xi	Jingang Wang	Tefeng Chen	Yifan Lu
Chi Zhang	Jiyuan He	Tianhao Bai	Yifan Zhang
Chong Peng	Jun Kuang	Tianhao Zhou	Yipeng Zang
Chuan Qin	Junhui Mei	Tingwen Xie	Yitao Zhai
Chuyu Zhang	Kai Liang	Wei He	Yiyang Li
Cong Chen	Ke He	Wei Huang	Yongjing Yin
Congkui Wang	Kefeng Zhang	Wei Liu	Yongkang Lv
Dan Ma	Keheng Wang	Wei Shi	Yongwei Zhou
Daoru Pan	Keqing He*	Wei Wang	Yu Yang
Defei Bu	Liang Gao	Wei Wu	Yuchen Xie
Dengchang Zhao	Liang Shi	Weikang Zhao	Yueqing Sun
Deyang Kong	Lianhui Ma	Wen Zan	Yuewen Zheng
Dishan Liu	Lin Qiu	Wenjie Shi	Yuhua Wei
Feiye Huo	Lingbin Kong	Xi Nan	Yulei Qian
Fengcun Li	Lingtong Si	Xi Su	Yunfan Liang
Fubao Zhang	Linkun Lyu	Xiang Li	Yunfang Tai
Gan Dong	Linsen Guo	Xiang Mei	Yunke Zhao
Gang Liu	Liqi Yang	Xiangyang Ji	Zeyang Yu
Gang Xu	Lizhi Yan	Xiangyu Xi	Zhao Zhang
Ge Li	Mai Xia	Xiangzhou Huang	Zhaohua Yang
Guoqiang Tan	Man Gao	Xianpeng Li	Zhenchao Zhang
Guoyuan Lin	Manyuan Zhang	Xiao Fu	Zhikang Xia
Haihang Jing	Meng Zhou	Xiao Liu	Zhiye Zou
Haomin Fu	Mengxia Shen	Xiao Wei	Zhizhao Zeng
Haonan Yan	Mingxiang Tuo	Xiaodong Cai	Zhongda Su
Haoxing Wen	Mingyang Zhu	Xiaolong Chen	Zhuofan Chen
Haozhe Zhao	Peiguang Li	Xiaoqing Liu	Zijian Zhang
Hong Liu	Peng Pei	Xiaotong Li	Ziwen Wang
Hongmei Shi*	Peng Zhao	Xiaowei Shi	Zixu Jiang
Hongyan Hao	Pengcheng Jia	Xiaoyu Li	Zizhe Zhao
Hongyin Tang	Pingwei Sun	Xili Wang	Zongyu Wang
Huantian Lv	Qi Gu	Xin Chen	Zunhai Su*
Hui Su	Qianyun Li	Xing Hu	LongCat-Flash
Jiacheng Li	Qingyuan Li*	Xingyu Miao	
Jiahao Liu	Qiong Huang	Xinyan He	

References

- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:arXiv preprint arXiv:2412.19437*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- Weilin Cai, Juyong Jiang, Le Qin, Junwei Cui, Sunghun Kim, and Jiayi Huang. Shortcut-connected expert parallelism for accelerating mixture-of-experts. *arXiv preprint arXiv:2404.05019*, 2024.
- Jiaming Wang, Yunke Zhao, Peng Ding, Jun Kuang, Zongyu Wang, Xuezhi Cao, and Xunliang Cai. Ask, fail, repeat: Meeseeks, an iterative feedback benchmark for llms’ multi-turn instruction-following ability. *arXiv preprint arXiv:2504.21625*, 2025a.
- Peng Jin, Bo Zhu, Li Yuan, and Shuicheng Yan. MoE++: Accelerating mixture-of-experts methods with zero-computation experts. *arXiv preprint arXiv:2410.07348*, 2024.
- Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.
- Zihao Zeng, Yibo Miao, Hongcheng Gao, Hao Zhang, and Zhijie Deng. AdaMoE: Token-adaptive routing with null experts for mixture-of-experts language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024a.
- Stuart Bennett. *A History of Control Engineering 1930-1955*. Peter Peregrinus, GBR, 1st edition, 1993. ISBN 0863412998.
- Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, 2022.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2017.
- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Yusuke Shibata, Takuya Kida, Shuichi Fukamachi, Masayuki Takeda, Ayumi Shinohara, Takeshi Shinohara, and Setsuo Arikawa. Byte pair encoding: A text compression scheme that accelerates pattern matching. 1999.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. Scaling exponents across parameterizations and optimizers. *arXiv preprint arXiv:2407.05872*, 2024.
- Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2net: Accelerating learning via knowledge transfer. *arXiv preprint arXiv:1511.05641*, 2015.
- Wenyu Du, Tongxu Luo, Zihan Qiu, Zeyu Huang, Yikang Shen, Reynold Cheng, Yike Guo, and Jie Fu. Stacking your transformers: A closer look at model growth for efficient LLM pre-training. *arXiv preprint arXiv:2405.15319*, 2024.
- Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. *arXiv preprint arXiv:2303.00980*, 2023a.
- Sheng Shen, Pete Walsh, Kurt Keutzer, Jesse Dodge, Matthew Peters, and Iz Beltagy. Staged training for transformer language models. In *International Conference on Machine Learning*, 2022.
- Yite Wang, Jiahao Su, Hanlin Lu, Cong Xie, Tianyi Liu, Jianbo Yuan, Haibin Lin, Ruoyu Sun, and Hongxia Yang. Lemon: Lossless model expansion. *arXiv preprint arXiv:2310.07999*, 2023b.
- Linyuan Gong, Di He, Zhuohan Li, Tao Qin, Liwei Wang, and Tieyan Liu. Efficient training of BERT by progressively stacking. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

- Dahyun Kim, Chanjun Park, Sanghoon Kim, Wonsung Lee, Wonho Song, Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Lee, Jihoo Kim, et al. Solar 10.7 b: Scaling large language models with simple yet effective depth up-scaling. *arXiv preprint arXiv:2312.15166*, 2023.
- Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.
- Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024.
- Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.
- Adrien Barbaresi. Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 2021.
- Xiangyu Xi, Deyang Kong, Jian Yang, Jiawei Yang, Zhengyu Chen, Wei Wang, Jingang Wang, Xunliang Cai, Shikun Zhang, and Wei Ye. Samplemix: A sample-wise pre-training data mixing strategy by coordinating data quality and diversity. *arXiv preprint arXiv:2503.01506*, 2025.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021a.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024b.
- Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. In *Advances in Neural Information Processing Systems*, 2023.
- Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Timothy Baldwin. CMMLU: Measuring massive multitask language understanding in chinese. *arXiv preprint arXiv:2306.09212*, 2023a.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- M-A-P Team, ByteDance. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. Challenging BIG-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. *arXiv preprint arXiv:1911.11641*, 2019.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019.
- Liang Xu, Hai Hu, Xuanwei Zhang, Lu Li, Chenjie Cao, Yudong Li, Yechen Xu, Kai Sun, Dian Yu, Cong Yu, Yin Tian, Qianqian Dong, Weitang Liu, Bo Shi, Yiming Cui, Junyi Li, Jun Zeng, Rongzhao Wang, Weijian Xie, Yanting Li, Yina Patterson, Zuoyu Tian, Yiwen Zhang, He Zhou, Shaowei Hua Liu, Zhe Zhao, Qipeng Zhao, Cong Yue, Xinrui Zhang, Zhengliang Yang, Kyle Richardson, and Zhenzhong Lan. CLUE: A Chinese language understanding evaluation benchmark. In *Proceedings of the 28th International Conference on Computational Linguistics*, 2020.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021b.
- Jiawei Liu, Songrun Xie, Junhao Wang, Yuxiang Wei, Yifeng Ding, and Lingming Zhang. Evaluating language models for efficient code generation. *arXiv preprint arXiv:2408.06450*, 2024b.
- Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022.
- Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024.
- Meta AI. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, 2025. URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- MoonshotAI. Kimi-K2 documentation, 2025. URL <https://moonshotai.github.io/Kimi-K2/>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Loubna Ben Allal, Niklas Muennighoff, Logesh Kumar Umapathi, Ben Lipkin, and Leandro von Werra. A framework for the evaluation of code generation models. <https://github.com/bigcode-project/bigcode-evaluation-harness>, 2022.
- Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. Scaling synthetic data creation with 1,000,000,000 personas. *arXiv preprint arXiv:2406.20094*, 2024.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.
- Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro von Werra, Arjun Guha, and Lingming Zhang. Selfcodealign: Self-alignment for code generation. In *Advances in Neural Information Processing Systems*, 2024.
- Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, 2024.
- Jin Jiang, Yuchen Yan, Yang Liu, Jianing Wang, Shuai Peng, Xunliang Cai, Yixin Cao, Mengdi Zhang, and Liangcai Gao. LogicPro: Improving complex logical reasoning via program-guided learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.
- Chenxu Wang, Ping Jian, and Zhen Yang. Thought-path contrastive learning via premise-oriented data augmentation for logical reading comprehension. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, 2025b.
- Junjie Ye, Caishuang Huang, Zhuohan Chen, Wenjie Fu, Chenyuan Yang, Leyi Yang, Yilong Wu, Peng Wang, Meng Zhou, Xiaolong Yang, et al. A multi-dimensional constraint framework for evaluating and improving instruction following in large language models. *arXiv preprint arXiv:2505.07591*, 2025.
- Yubo Wang, Xiang Yue, and Wenhui Chen. Critique fine-tuning: Learning to critique is more effective than learning to imitate. *arXiv preprint arXiv:2501.17703*, 2025c.
- Tong Mu, Alec Helyar, Johannes Heidecke, Joshua Achiam, Andrea Vallone, Ian Kivlichan, Molly Lin, Alex Beutel, John Schulman, and Lilian Weng. Rule based rewards for language model safety. In *Advances in Neural Information Processing Systems*, 2024.
- Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E Gonzalez, and Ion Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint arXiv:2406.11939*, 2024a.
- Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E Gonzalez, and Ion Stoica. From live data to high-quality benchmarks: The arena-hard pipeline, April 2024b. URL <https://lmsys.org/blog/2024-04-19-arena-hard/>.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

- Shunyu Yao, Howard Chen, Austin W. Hanjie, Runzhe Yang, and Karthik R Narasimhan. COLLIE: Systematic construction of constrained text generation tasks. In *The Twelfth International Conference on Learning Representations*, 2024.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.
- MAA. Aime 2024, 2024. URL <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>.
- MAA. Aime 2025, 2025. URL <https://artofproblemsolving.com/wiki/index.php/AIMEProblemsandSolutions>.
- ByteDance-Seed. Beyondaime: Advancing math reasoning evaluation beyond high school olympiads. <https://huggingface.co/datasets/ByteDance-Seed/BeyondAIME>, 2025.
- Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. ZebraLogic: On the scaling limits of LLMs for logical reasoning. In *Forty-second International Conference on Machine Learning*, 2025.
- OpenAI. Graphwalks dataset, 2025a. URL <https://huggingface.co/datasets/openai/graphwalks>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- The Terminal-Bench Team. Terminal-bench: A benchmark for ai agents in terminal environments, Apr 2025a. URL <https://github.com/laude-institute/terminal-bench>.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Yuefeng Huang, et al. ACEBench: Who wins the match point in tool learning? *arXiv preprint arXiv:2501.12851*, pages arXiv–2501, 2025.
- OpenAI. Introducing GPT-4.1 in the api, April 2025b. URL <https://openai.com/index/gpt-4-1/>.
- Anthropic. Introducing claude 4, May 2025. URL <https://www.anthropic.com/news/claude-4>.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Author Qi and Others. Zero-bubble pipeline parallelism for large language models. *arXiv preprint arXiv:2301.12345*, 2023.
- Penghui Qi, Xinyi Wan, Nyamdavaa Amar, and Min Lin. Pipeline parallelism with controllable memory. 2024.
- Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. NanoFlow: Towards optimal large language model serving throughput. *arXiv preprint arXiv:2408.12757*, 2025.
- Yulei Qian, Fengcun Li, Xiangyang Ji, Xiaoyu Zhao, Jianchao Tan, Kefeng Zhang, and Xunliang Cai. EPS-MoE: Expert pipeline scheduler for cost-efficient moe inference. *arXiv preprint arXiv:2410.12247*, 2025.
- The SGLang Team. Deploying deepseek with pd disaggregation and large-scale expert parallelism on 96 h100 gpus. <https://lmsys.org/blog/2025-05-05-large-scale-ep/>, 2025b. Accessed: [May 2025].
- Jack Choquette. Nvidia hopper gpu: Scaling performance. In *2022 IEEE Hot Chips 34 Symposium (HCS)*, 2022.
- Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. *arXiv preprint arXiv:2408.11049*, 2025.
- Jiahao Liu, Qifan Wang, Jingang Wang, and Xunliang Cai. Speculative decoding via early-exiting for faster llm inference with thompson sampling control mechanism. *arXiv preprint arXiv:2406.03853*, 2024c.

- Feiye Huo, Jianchao Tan, Kefeng Zhang, Xunliang Cai, and Shengli Sun. C2T: A classifier-based tree construction method in speculative decoding. *arXiv preprint arXiv:2502.13652*, 2025.
- Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.
- Chenggang Zhao, Liang Zhao, Jiashi Li, and Zhean Xu. DeepGEMM: clean and efficient fp8 gemm kernels with fine-grained scaling. <https://github.com/deepseek-ai/DeepGEMM>, 2025a.
- Pengcui Dege, Qiuming Luo, Rui Mao, and Chang Kong. FlashMLA-ETAP: Efficient transpose attention pipeline for accelerating mla inference on nvidia h20 gpus. *arXiv preprint arXiv:2506.01969*, 2025.
- NVIDIA. NVIDIA Collective Communications Library (NCCL). <https://github.com/NVIDIA/nccl>. Version 2.21.5.
- Aashaka Shah, Abhinav Jangda, Binyang Li, Caio Rocha, Changho Hwang, Jithin Jose, Madan Musuvathi, Olli Saarikivi, Peng Cheng, Qinghua Zhou, Roshan Dathathri, Saeed Maleki, and Ziyue Yang. MSCCL++: Rethinking gpu communication abstractions for cutting-edge ai applications. *arXiv preprint arXiv:2504.09014*, 2025.
- Qingyuan Li, Yifan Zhang, Liang Li, Peng Yao, Bo Zhang, Xiangxiang Chu, Yerui Sun, Li Du, and Yuchen Xie. FPTQ: Fine-grained post-training quantization for large language models. *arXiv preprint arXiv:2308.15987*, 2023b.
- Zunhai Su, Qingyuan Li, Hao Zhang, YuLei Qian, Yuchen Xie, and Kehong Yuan. Unveiling super experts in mixture-of-experts large language models. *arXiv preprint arXiv:2507.23279*, 2025.
- Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025b.
- DeepSeek. Profiling data in deepseek infra. <https://github.com/deepseek-ai/profile-data>, 2025a. Accessed: [May 2025].
- DeepSeek. Day 6: One more thing, deepseek-v3/r1 inference system overview. https://github.com/deepseek-ai/open-infra-index/blob/main/2025020OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md, 2025b. Accessed: [May 2025].
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.
- Shengyu Liu Jiashi Li. FlashMLA: Efficient mla decoding kernels. <https://github.com/deepseek-ai/FlashMLA>, 2025.

A Appendix

A.1 Statistics and Case Studies of Dynamic Routing

Figure 11 shows the average activated FFN experts of LongCat-Flash base model across benchmarks. A consistent computational bias favors English tokens over Chinese and mathematical ones. We present a more detailed expert

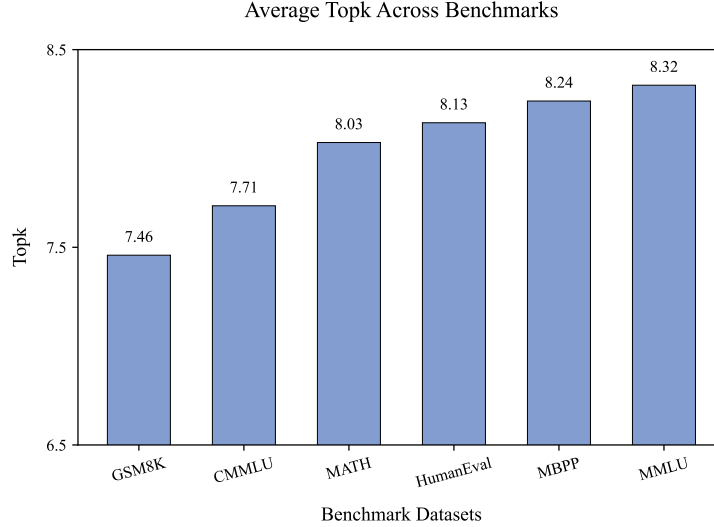


Figure 11: The average number of activated FFN experts across different benchmarks.

selection across different layers for several cases in Table 8. These cases reveal different patterns of expert selection across layers. In the first layer, function words (including articles, conjunctions, prepositions), numbers and punctuation marks consistently receive lower computational resources. In contrast, the final layer (Layer 28) exhibits less specialized feature allocation compared to Layer 1, though identifiable patterns still exist. For example, in the Chinese text case, tokens preceding punctuation marks tend to be assigned fewer computational resources. We hypothesize that shallow layers prioritize token-internal semantics for allocation, while deeper layers dynamically adjust resources based on predictive complexity, potentially reflecting a hierarchical transition from local feature processing to global prediction optimization.

Layer 1 - English A large language model (LLM) is a language model trained with self-supervised machine learning on a vast amount of text, designed for natural language processing tasks, especially language generation. The largest and most capable LLMs are generative pretrained transformers (GPTs), based on a transformer architecture, which are largely used in generative chatbots such as ChatGPT, Gemini and Claude. LLMs can be fine-tuned for specific tasks or guided by prompt engineering. These models acquire predictive power regarding syntax, semantics, and ontologies inherent in human language corpora, but they also inherit inaccuracies and biases present in the data they are trained on.
Layer 1 - Math Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\nQuestion: A robe takes 2 bolts of blue fiber and half that much white fiber. How many bolts in total does it take?\nLet's think step by step.\nAnswer: It takes $2/2 = 1$ bolt of white fiber So the total amount of fabric is $2+1=3$ bolts of fabric The answer is 3.\n\nQuestion: In a dance class of 20 students, 20% enrolled in contemporary dance, 25% of the remaining enrolled in jazz dance, and the rest enrolled in hip-hop dance. What percentage of the entire students enrolled in hip-hop dance?\nLet's think step by step.
Layer 1 - Code <pre>def quick_sort(arr: list) -> list:\n if len(arr) <= 1:\n return arr\n pivot = arr[len(arr) // 2]\n left = [x for x in arr if x < pivot]\n middle = [x for x in arr if x == pivot]\n right = [x for x in arr if x > pivot]\n return quick_sort(left) + middle + quick_sort(right)\n\nif __name__ == "__main__":\n unsorted_list = [3, 6, 8, 10, 1, 2, 1, 5, 7, 4]\n sorted_list = quick_sort(unsorted_list)\n print(sorted_list)</pre>
Layer 1 - Chinese 宇宙学中通常所说的“大爆炸”是指：宇宙是在过去有限的时间之前，由一个密度极大且温度极高的太初状态 (Initial singularity) 演变而来的。根据2015年普朗克卫星所得到的最佳观测结果，宇宙大爆炸距今 137.99 ± 0.21 亿年，并经过不断的膨胀到达今天的状态。大爆炸这一模型的框架基于爱因斯坦的广义相对论，又在场方程的求解上作出了一定的简化（例如宇宙学原理假设空间的均匀性和各向同性）。1922年，苏联物理学家亚历山大·弗里德曼用广义相对论描述了流体，从而给出了这一模型的场方程。1927年，比利时物理学家乔治·勒梅特通过求解弗里德曼方程已经在理论上提出了同样的观点，这个解后来被称作弗里德曼-勒梅特-罗伯逊-沃尔克度规。
Layer 28 - English A large language model (LLM) is a language model trained with self-supervised machine learning on a vast amount of text, designed for natural language processing tasks, especially language generation. The largest and most capable LLMs are generative pretrained transformers (GPTs), based on a transformer architecture, which are largely used in generative chatbots such as ChatGPT, Gemini and Claude. LLMs can be fine-tuned for specific tasks or guided by prompt engineering. These models acquire predictive power regarding syntax, semantics, and ontologies inherent in human language corpora, but they also inherit inaccuracies and biases present in the data they are trained on.
Layer 28 - Math Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\nQuestion: A robe takes 2 bolts of blue fiber and half that much white fiber. How many bolts in total does it take?\nLet's think step by step.\nAnswer: It takes $2/2 = 1$ bolt of white fiber So the total amount of fabric is $2+1=3$ bolts of fabric The answer is 3.\n\nQuestion: In a dance class of 20 students, 20% enrolled in contemporary dance, 25% of the remaining enrolled in jazz dance, and the rest enrolled in hip-hop dance. What percentage of the entire students enrolled in hip-hop dance?\nLet's think step by step.
Layer 28 - Code <pre>def quick_sort(arr: list) -> list:\n if len(arr) <= 1:\n return arr\n pivot = arr[len(arr) // 2]\n left = [x for x in arr if x < pivot]\n middle = [x for x in arr if x == pivot]\n right = [x for x in arr if x > pivot]\n return quick_sort(left) + middle + quick_sort(right)\n\nif __name__ == "__main__":\n unsorted_list = [3, 6, 8, 10, 1, 2, 1, 5, 7, 4]\n sorted_list = quick_sort(unsorted_list)\n print(sorted_list)</pre>
Layer 28 - Chinese 宇宙学中通常所说的“大爆炸”是指：宇宙是在过去有限的时间之前，由一个密度极大且温度极高的太初状态 (Initial singularity) 演变而来的。根据2015年普朗克卫星所得到的最佳观测结果，宇宙大爆炸距今 137.99 ± 0.21 亿年，并经过不断的膨胀到达今天的状态。大爆炸这一模型的框架基于爱因斯坦的广义相对论，又在场方程的求解上作出了一定的简化（例如宇宙学原理假设空间的均匀性和各向同性）。1922年，苏联物理学家亚历山大·弗里德曼用广义相对论描述了流体，从而给出了这一模型的场方程。1927年，比利时物理学家乔治·勒梅特通过求解弗里德曼方程已经在理论上提出了同样的观点，这个解后来被称作弗里德曼-勒梅特-罗伯逊-沃尔克度规。

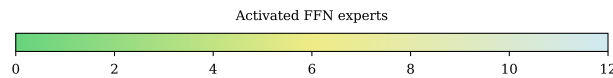


Table 8: The number of activated FFN experts per token across layers.